

(Advanced) Cloud Prototyping
– Trusted Real-Time Assessment of Chained Emissions –
Trace

Written by:	Felix Hasel	480722
	Yunusemre Kilic	483878
	Reine Labes	469254
	Ramon Mehrpoya	371968
	Greta Möller	402332
	Van Thang Nguyen	519871
	Frederic Rupprecht	519879
	Lucas Stuve	412408

Supervised by:	Dr.-Ing. Stefan Tai
	Dr.-Ing. Jonathan Johannes Heiß
	M. Sc. Jannis Kiesel

Version from:	October 26, 2025
----------------------	------------------

Abstract

The increasing regulatory pressure and public demand for transparent carbon accounting in global supply chains presents a fundamental challenge: how to verify Product Carbon Footprint (PCF) calculations while protecting sensitive business information? This report presents *TRACE* (Trusted Real-Time Assessment of Chained Emissions), a prototypical implementation of a verifiable carbon accounting system for transport chains that resolves this tension through the integration of Zero-Knowledge Proof (ZKP)s and business process management. *TRACE* addresses the limitations of traditional trust-based carbon reporting systems by enabling mathematical verification of PCF calculations without disclosing confidential operational data. The system implements a process-centric approach using Business Process Model and Notation (BPMN)-based workflow orchestration through Camunda, combined with cryptographic verification using *RISC Zero*'s Zero-Knowledge Virtual Machine (zkVM). By adhering to established industry standards including Integrating Logistics Emissions and PCFs (iLEAP), Global Logistics Emissions Council (GLEC), Partnership for Carbon Transparency (PACT), and ISO 14083, *TRACE* ensures interoperability while introducing verifiable computation capabilities. The architecture employs a microservice design with clearly separated private and public domains, enabling companies to maintain control over sensitive data while participating in transparent carbon accounting networks. A key optimization is our adoption and adaptation of lazy signature verification from Janwillem Swalens et al. to the carbon accounting context, which significantly reduces the computational overhead of proof generation - a critical bottleneck in ZKP systems. Performance benchmarking demonstrates that nested proof composition scales linearly with chain depth concerning proof size while maintaining constant proving time per calculation, making the approach viable for real-world transport scenarios with multiple participants. Evaluation results confirm that *TRACE* successfully fulfills its core objectives: enabling standards-compliant PCF calculations, providing cryptographic verification without compromising business confidentiality, and achieving practical performance for multi-party supply chains. While certain limitations remain, particularly regarding completeness verification and the need for trusted auditors in specific scenarios, *TRACE* demonstrates the feasibility of transitioning from trust- or transparency-based to mathematically verifiable carbon accounting systems in the transportation sector.

TABLE OF CONTENTS

1	Motivation	1
2	Background	3
2.1	Product Carbon Footprinting (PCF)	3
2.2	Business Process Management	4
2.2.1	Introduction to Business Process Management	4
2.2.2	Workflow Engines	4
2.2.3	Camunda Platform	5
2.3	Zero-Knowledge Proofs	5
2.4	Risc Zero and zkVM	6
2.5	iLEAP and Emission Data Standards	7
2.5.1	Foundational Standards and Protocol	7
2.5.2	Data Structures	9
3	Goals and Assumptions	10
3.1	System Goals	10
3.2	Assumptions	11
4	Related Work	12
5	System Design	15
5.1	Scenario	15
5.2	Architecture Overview	16
5.3	Service Components	17
5.3.1	Business Process Service	17
5.3.2	Proving Service	18
5.3.3	Sensor Data Service	18
5.3.4	Sensor Key Registry	18
5.3.5	Verifier Service	19
5.3.6	PCF Registry	19
5.3.7	Kafka Messaging Infrastructure	19
5.3.8	Proof Program Registry	20
5.4	Component Interaction	20
5.5	Integrating and extending existing standards	21
5.6	The Role of Auditors	22

5.7	Required Setup and Configuration	23
5.8	Trust System	25
6	Implementation	27
6.1	Business Process Service	27
6.1.1	Process Flow Example: Amazing Company 2	28
6.2	PCF Calculation and Proof Generation	30
6.2.1	Sensor Data Signatures	31
6.2.2	Proving Service	31
6.3	Deployment	33
6.4	Runtime Operation	34
7	Benchmarking & Methodology	35
7.1	Metrics	35
7.2	Benchmarking setup	36
7.2.1	Hardware specifications	36
7.2.2	Software specifications	36
7.3	Experiments and Rationale	36
7.3.1	Methodology	36
7.3.2	Experiment	37
8	Evaluation	38
8.1	Proof Generation Performance	38
8.2	Trust assumptions lazy signature verification	41
9	Conclusion and Envisioned Future	46
9.1	Conclusion	46
9.2	Limitations	46
9.2.1	Threats to Validity Discussion	47
9.3	Future Work	49
9.3.1	Proof Program Verification Service	49
9.3.2	Traceability	50
9.3.3	Selective Disclosure	51
9.3.4	Plausibility Checks	51
9.3.5	Verification of Emission Operation Categories	51
A	Abbreviations	54
B	Documents, Data Examples and AI Generated Content	55
B.1	Proving Document	55
B.2	ProofResponse	58
B.3	Proving Service Pseudo Code	58
B.4	Use/Misuse overview	60
B.5	AI Generated Prompts	62
B.6	Project Contributions	62

Chapter 1

Motivation

The necessity of accurately capturing and transparently passing carbon footprints of products along global supply chains is undisputed in view of regulatory developments and growing public pressure. The so-called *PCF* is a key figure that is aggregated over all life cycle phases, from raw material extraction to disposal. Initiatives like the *Greenhouse Gas (GHG) Protocol Standard*¹, the *GLEC Framework*² and regulatory developments at EU level (e.g. Regulation (EU) 2024/1781³) increasingly influence the calculation and disclosure of emission data.

A central problem of conventional PCF systems is their lack of verifiability and the resulting dependence on trust-based information from individual actors. As Heiß et al. (2024) [8] explain in their paper “Verifiable Carbon Accounting in Supply Chains”, there is a structural tension, particularly in complex supply chains, between the need to disclose emissions-relevant data and the legitimate interest in protecting trade secrets. The presented approach of *Verifiable Carbon Accounting (VCA)* addresses this problem by integrating cryptographic procedures like *ZKPs* in existing *Monitoring-Reporting-Verification (MRV)* infrastructures. A central characteristic of this approach is the possibility to mathematically prove the correctness of emission-based calculations and data flows without disclosing confidential data.

This conflict of trust and confidentiality is also reflected in the EU *Regulation on the accounting of greenhouse gas emissions of transport services*⁴, in force since 2025. The regulation seeks to improve the reliability of emission assessments by encouraging the use of standards such as GLEC and ISO 14083:2023, while introducing verification mechanisms under the European Environment Agency. To balance costs, extensive obligations apply mainly to large companies.

In addition to advances in research, there are various industry initiatives that establish practical standards for data exchange. These promote standardisation and cross-company acceptance of data exchange, thereby supporting carbon reporting. This is also the case in the transportation sector, which seems to be a suitable test field for carbon reporting systems. On the one hand, initiatives such as iLEAP ensure uniform data standards, and on the other hand, business activities are comparatively simple. For example, the fuel consumption of a transport service can be used directly for emission calculation, as it is easy to measure and trace. Furthermore, a process-oriented approach supports establishment in corporate practice. Re-

¹<https://ghgprotocol.org/>

²<https://www.smartfreightcentre.org/en/tools/glec-framework/>

³<https://eur-lex.europa.eu/eli/reg/2024/1781/oj/eng>

⁴[https://www.europarl.europa.eu/RegData/etudes/BRIE/2023/753193/EPRS_BRI\(2023\)753193_EN.pdf](https://www.europarl.europa.eu/RegData/etudes/BRIE/2023/753193/EPRS_BRI(2023)753193_EN.pdf)

porting is integrated directly into the company’s technical and process management workflows. Based on the process data collected by companies, the granularity and traceability of emissions calculations along the value chain can be increased.

These efforts to establish uniform standards form a valuable basis for cooperation in the supply chain to support carbon reporting. However, based on current knowledge, these are not sufficient to support cooperation in a supply chain characterized by conflicts of interest. Due to mutual mistrust, an independent party must therefore be appointed to guarantee compliance with the standards at a cost.

The project *TRACE* (Trusted Real-time Assessment of Chained Emissions) is intended as a prototypical implementation and extension of the concept presented by Heiß et al. *TRACE* incorporates process-centric modeling (via BPMN), modular microservice architectures, and cryptographically verifiable execution of emissions calculations using *zkVMs* (specifically *RISC Zero*). Its core objective is to enable all partners to calculate their Product Carbon Footprint using established standards while providing mathematically verifiable carbon accounting. A central aim is to resolve the tension between data transparency and business confidentiality, allowing for verification without exposing sensitive trade secrets. This approach is designed to minimize reliance on traditional trust mechanisms and audits, thus reducing cost and complexity. To ensure practical adoption, the system will utilize a process-centric approach embedded in existing business workflows, guarantee the authenticity of sensor data, and achieve scalable performance for realistic scenarios. Ultimately, *TRACE* will demonstrate that verifiable, confidential carbon accounting can be integrated across multi-party supply chains, serving as a proof of concept for a trustworthy digital infrastructure that supports interoperability and formal correctness in carbon reporting across organizational boundaries.

Chapter 2

Background

To understand the TRACE system, an understanding of the central concepts on which it is based is necessary. This chapter therefore introduces the most important technical and business principles relevant for the development of a verifiable, process-centric system for calculating CO₂ emissions along transport chains.

2.1 Product Carbon Footprinting (PCF)

PCF is the total amount of greenhouse gas emissions (usually in CO₂ equivalents) for a product's life cycle from cradle to grave. The complete lifecycle represents all activities from raw material production until end-of-life disposal. The intention behind the PCF is to make a product's climate footprint measurable, comparable, and transparent. Since the 1990s, companies have upped their examination of their goods and processes from the perspective of their GHG emissions [11]. The intent is to identify both internal cost-savings opportunities and to report them externally to stakeholders (investors, customers, regulators)¹ [6]. PCF is therefore becoming increasingly important both to corporate management and public interest.

Historically, the PCF idea developed in the context of life cycle assessment (product life cycle analysis). In view of comparison, there has been a stepwise introduction of consistent calculation requirements. The most important of them are ISO 14067 and the GHG Protocol Product Standard². The requirements specify the boundaries of the assessment system (e.g., "cradle-to-gate" or "cradle-to-grave"), whether direct and indirect emissions (Scopes 1 to 3) are included, and specify data quality, transparency, and requirements for documenting assumptions, uncertainties, and data sources [10]. Presentation of the results is also managed to enable comparability.

In recent years, developments in the field of PCF assessment have focused on three key objectives:

- Improving the data basis through increased use of primary data instead of averages³,
- Automating and increasing the efficiency of calculations through digitization and integration into business processes,

¹<https://www.mckinsey.com/capabilities/operations/our-insights/product-sustainability-back-to-the-drawing-board>

²<https://ghgprotocol.org/>

³<https://www.idhsustainabletrade.com/idh-environmental-footprint-roadmap/>

- Increasing the interoperability of emissions data through common exchange formats and standards.

Another salient issue is indirect emissions verifiability (Scope 3): Apart from ensuring they accurately compute their own values, companies must determine whether they can rely on third-party reported emissions, ideally without knowing every detail of their internal workings. Such trends are echoed, for example, in recent World Bank reports on further development of MRV systems to digital D-MRV systems [19]. Industry-driven research solutions such as Siemens' SiGreen also demonstrate the trend toward traceable, automated verification of emissions along the supply chain [15].

TRACE surmounts these barriers by enabling process-based structural computation of PCF values along logistics chains. All partners calculate the PCF for their share of the supply chain. The results are exchanged in the form of digital, cryptographically signed receipts and are verifiable by third parties without revealing internal information.

2.2 Business Process Management

The following sections discuss Business Process Management (BPM), Workflow Engines and the Camunda platform.

2.2.1 Introduction to Business Process Management

BPM is a systematic approach to making organizations' workflows more effective, efficient and capable of adapting to an ever-changing environment. This management discipline includes concepts, methods, and techniques to support the design, administration, configuration, enactment, and analysis of business processes. Moreover, BPM is fundamentally based on the explicit representation of business processes, including their activities and the execution constraints between them. Moreover, by utilizing the BPMN, BPM bridges the gap between business professionals and IT staff. BPMN provides standardized elements, such as business activities and execution rules (e.g. gateways), that enable professionals to document their processes in a way that can be easily understood and implemented by IT within the company's systems. Once defined, these processes can then be analyzed, improved, and enacted [18].

2.2.2 Workflow Engines

A Workflow Engine is a central technical component in the implementation of BPM, responsible for modeling, executing and managing business processes. It interprets defined workflows (BPMN) and reliably orchestrates the execution of tasks by specified roles, services or systems at the appropriate point in the sequence. A key feature of modern Workflow Engines is the decoupling of process logic from application logic, which allows for greater flexibility and reusability in business process implementations. Process definitions are written as process models, stored separately from application code, and run by the workflow engine at runtime. This allows processes to be easily reused, adapted, and examined without extensive manipulation of the application core [18].

2.2.3 Camunda Platform

Camunda⁴ is a widely adopted open-source workflow and decision automation platform with native support for BPMN 2.0. It allows organizations to manage processes based on their BPMN models, connect them with external logic, and monitor them extensively. Camunda has a modular architecture and supports both on-premise deployments as well as cloud-native environments. Camunda provides a modular set of tools⁵ :

- Zeebe: A scalable, cloud-native workflow engine that orchestrates processes, manages their state, and executes activities with low latency.
- Camunda Modeler: A graphical user interface for editing and creating BPMN process models.
- Camunda Operate: Offers monitoring and debugging capabilities for executing processes.
- Camunda Tasklist: Facilitates combining manual user input with automated processes.
- Camunda Optimize: Offers analytics functionality to detect bottlenecks and optimize processes according to data.

These elements provide complete control and monitoring of business processes. Integration capabilities in Camunda are facilitated through its external task pattern, which enables loose coupling between the workflow engine and external service components. This architectural approach allows organizations to implement process activities using their preferred technologies and programming languages while maintaining coordination through the central workflow engine. External workers can be scaled independently based on their specific performance requirements, and new workers can be added or removed without affecting the core workflow engine or other system components.

2.3 Zero-Knowledge Proofs

ZKPs are cryptographic protocols by which a party (the prover) is able to demonstrate to another party (the verifier) that a specific statement is true without disclosing additional information [7, 13]. A zero-knowledge proof fulfills three significant properties [5]:

- Completeness: if the statement is true, an honest prover can always convince the verifier.
- Soundness: if the statement is false, an unethical prover has only a vanishing chance of deceiving the verifier.
- Zero-Knowledge: in the limited sense, the verifier learns nothing about the underlying information or the exact computation process, but only that the statement is true.

For actual use, several kinds of ZKPs exist, each varying in terms of efficiency, trust assumptions, proof size and verification time. Most widely used are two underlying proof systems: Succinct Non-Interactive Arguments of Knowledge (zk-SNARK)s and Scalable Transparent Arguments of Knowledge (zk-STARK)s. zk-SNARKs offer very short proofs and fast,

⁴<https://camunda.com/>

⁵<https://camunda.com/platform/>

non-interactive verification but come with a so-called “trusted setup” protocol and make use of elliptic curve cryptography, which could potentially be vulnerable to quantum computers in the future, zk-STARKs, on the other hand, do not require a trusted setup, use hash functions as their security foundation, and therefore are more transparent and potentially quantum-attack-resistant [16]. However, they produce larger proofs and are therefore more demanding in terms of storage, transmission, and computational power due to the increased complexity of the proof generation and verification process.

ZKPs are especially relevant in scenarios that demand both verifiable computation and strong confidentiality, such as in high-stakes business processes, finance, supply chains, or blockchain. With the unification of data protection and mathematically assured verifiability, they offer a means of proving and trust-building between independent parties without disclosing internal information.

2.4 Risc Zero and zkVM

Traditionally, implementations have relied on Domain-Specific Languages (DSLs) to define zero-knowledge circuits, offering fine-grained control over performance and logic. However, this approach can require specialized knowledge. *RISC Zero* lowers the entry barrier by providing a zkVM that supports widely used programming languages and well-established toolchains⁶. Instead of manually encoding ZK circuits, *RISC Zero* authors write regular Rust code and compile it.

The *RISC Zero* architecture is structured into two distinct components: guest code and host code. The guest code runs within the zkVM and represents the prover’s role, performing the computation and generating a proof in the form of a receipt. In contrast, the host code operates on the verifier’s side. It interfaces with external systems, invokes the guest code with inputs, initiates the zkVM, validates the outputs, and may perform additional processing on the data contained in the receipt.

This receipt consists of the journal and the seal. The journal contains all values annotated as public, and also optionally other results specified by the guest code. The seal is the actual zero-knowledge proof, which certifies that journal data is from the execution of the provided program⁸.

The proof process shown in fig. 2.1 begins with compiling the source code written in Rust into a RISC-V ELF binary. This binary is then executed within the zkVM in an arithmetic circuit-based riscv32im instruction set implementation, with the execution logged in full.

The verification, later, if performed, is based on this execution log, and a seal/inner receipt is generated. If the hash of the data contained in the proof exactly matches the expected hash of the underlying memory image, the calculation is verified as correct.

It is common practice to measure the effort required for ZKP processes (proof generation and verification) in CPU cycles. The more cycles, the larger the final proof and the computation cost involved⁹. The computational effort in producing a *RISC Zero* proof is significant and should not be underestimated in practical applications. Verifying a proof is much faster in comparison,

⁶<https://dev.risczero.com/>

⁸https://docs.rs/risc0-zkvm/latest/risc0_zkvm/struct.Receipt.html#method.verify

⁹<https://dev.risczero.com/api/zkvm/optimization>

Building on this foundation, ISO 14083¹⁰ [2] codifies the GLEC methodology into a formal, internationally recognized standard. It specifies the principles and requirements for quantifying and reporting emissions from any transport chain, thereby enhancing the comparability, credibility, and transparency of reported data for all stakeholders, including shippers, carriers, and logistics service providers.

While ISO 14083 defines what to calculate, PACT¹¹ focuses on how to exchange this information. Through its Pathfinder Framework, PACT provides a common technical language and data protocol for sharing product-level carbon footprint data, ensuring that emissions data calculated according to ISO 14083 can be seamlessly shared and understood across supply chain partners.

The most indispensable standard for our work is the iLEAP¹² Technical Specification. While the preceding standards provide the "what" and "how", iLEAP provides the implementation layer that makes large-scale, automated data exchange feasible. It directly addresses the operational bottleneck of manual data sharing by defining a standardized, automatable protocol. The PACT data scheme offers a field for extensions. By providing such an extension to the PACT data schemes, iLEAP makes the principles of ISO 14083 actionable at scale, which is a central requirement for our prototype.

By combining all four, the methodology, formal standardization, interoperable data structure, and automated transfer mechanism reinforce each other, creating an end-to-end system capable of delivering trustworthy emissions data across complex, multi-party supply chains.

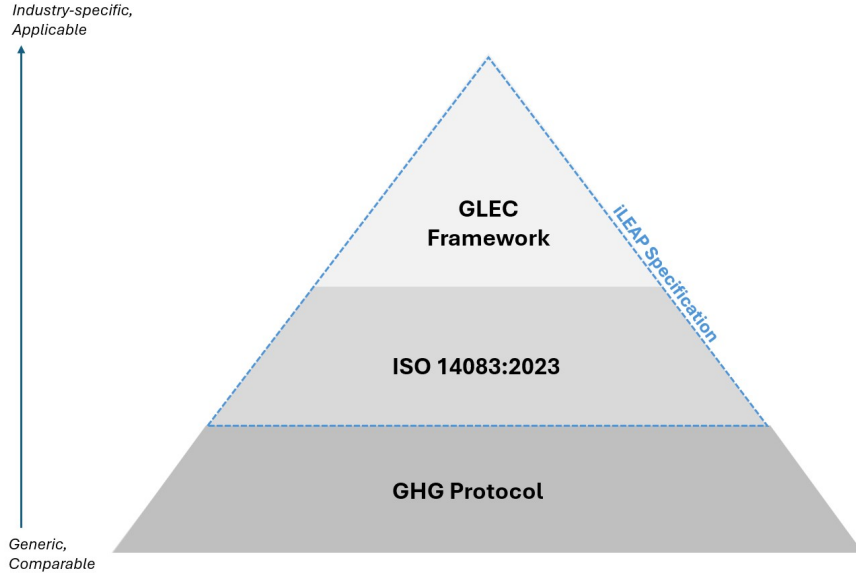


Figure 2.2: Interplay of the different Standards, Frameworks, Specifications in the Context of Carbon Reporting of Transport Services

¹⁰<https://www.iso.org/contents/news/2023/01/a-net-zero-logistics-sector.html>

¹¹<https://docs.carbon-transparency.org/tr/data-exchange-protocol/latest/>

¹²<https://sine-fdn.github.io/ileap-extension/#introduction>

2.5.2 Data Structures

To consistently quantify and report emissions, the GLEC Framework and ISO 14083 establish a standardized vocabulary and a hierarchical data structure. These concepts define the actors involved, the activities being measured, and the groupings used for analysis and reporting. PACT defines the exchange of that data and iLEAP provides an extensions for the PACT data schemes.

At the highest level is the *Shipment*, which refers to the specific goods being moved in a commercial transaction between a consignor (seller) and a consignee (buyer). The entire journey of a shipment is defined as a Transport Chain. A Transport Chain is not a monolithic entity; rather, it is modeled as a sequence of discrete components.

Each component in the sequence is called a *Transport Chain Element (TCE)*. A TCE represents a distinct stage of the journey and can be either a movement or a stop:

1. **Transport Activity:** This represents the movement of goods. Its magnitude is calculated by multiplying the mass of the consignment by the distance it travels, typically expressed in tonne-kilometers. This is the fundamental unit of work against which emissions from movement are calculated.
2. **Hub Activity:** This represents a stationary period at a facility (e.g., a warehouse, port, or distribution center) where operations like loading, unloading, sorting, or temporary storage occur. The activity is measured by the facility's throughput relevant to the shipment.

For practical calculation and reporting, these individual activities are grouped into standardized categories based on their characteristics. This aggregation is crucial for developing reliable emission factors when highly specific, primary data is unavailable. Using primary measurements for each data point is not feasible, which is another reason why categories are determined to group similar activities.

A *Transport Operation Category (TOC)* is a grouping of transport activities with similar attributes. For example, a TOC could be defined as "road freight using a 40-tonne truck in Western Europe" or "transpacific container shipping." By grouping similar operations, an average emissions performance (e.g., grams of CO₂-e per tonne-kilometer) can be determined.

Similarly, a *Hub Operation Category (HOC)* is a grouping of hub operations with similar characteristics, such as "refrigerated warehousing" or "automated parcel sorting hub".

The data for TOC and HOC categories is determined historically. A transport operator or hub manager analyzes their operational data (e.g., total fuel consumed, total tonne-kilometers performed, total goods processed) over a defined reporting period, such as a full calendar year. From this historical data, they calculate an average emission factor for that category. This calculated factor is considered valid and can be used for reporting and planning purposes for the subsequent period, after which it should be recalculated with new operational data to reflect changes in efficiency, equipment, or fuel types.

Finally, the standards define the roles of the actors within this ecosystem: The *Transport Operator*, also called the contractor from now on, is the party that physically executes the transport service (e.g., the trucking company, airline, or shipping line). The *Transport Service Organizer*, also called transport service provider, is the party that arranges the transport service, which may involve subcontracting the physical work to one or more Transport Operators (e.g., a freight forwarder or 4PL provider).

Chapter 3

Goals and Assumptions

This chapter outlines the goals for our prototype as well as the basic assumptions of the *TRACE* system for verifiable calculation of Product Carbon Footprints along transport chains. The goals are based on the scientific principles from the verifiability of cryptographic systems, regulatory requirements from carbon accounting standards and discussions with the SINE foundation, leading experts in the field of carbon reporting.

3.1 System Goals

The following system goals are presented in a prioritized order, with foundational requirements stated first to emphasize their relative significance for successful implementation and usage.

- G. 1** Calculate Product Carbon Footprint: Enable all supply chain partners to calculate their PCF using established industry standards.
- G. 2** Provide Verifiable Carbon Accounting: Implement a system that allows mathematical verification of PCF calculations.
- G. 3** Utilize Process-Centric Approach: Embed verifiable carbon accounting (Goal 2) into standard business process management systems to ensure adoption within existing business infrastructure rather than requiring parallel systems.
- G. 4** Protect Business Confidentiality: Resolve the tension between emission data transparency requirements and legitimate trade secret protection, which enables regulatory compliance and supply chain transparency while maintaining competitive advantages and protecting sensitive business information.
- G. 5** Ensure Sensor Data Authenticity: Guarantee the provenance and tamper-proof integrity of sensor data used in PCF calculations.
- G. 6** Minimize Trust Requirements: Reduce dependence on traditional trust mechanisms and audit processes, resulting in lower cost and complexity of carbon accounting verification while increasing reliability.
- G. 7** Achieve Scalable Performance: Ensure the system can handle realistic supply chain scenarios without prohibitive computational overhead, demonstrating that cryptographic verification approaches are practical for real-world deployment.

- G. 8** Demonstrate Multi-Party Supply Chain Integration: Show that verifiable carbon accounting (Goal 2) can work across multi-party supply chains, enabling verification of aggregate carbon footprint calculations across organizational boundaries.

Together, these goals aim to establish a secure, scalable, and verifiable system for carbon accounting that balances transparency, confidentiality, and practical applicability across supply chains.

3.2 Assumptions

In order to scope the project and to not detail edge cases, we make the following assumptions on the scenario the *TRACE* system is used in.

- A. 1** Initial setup: Participating companies have completed an initial setup. This includes communication channels, sharing access rights to selected data exchange points, agreement on standards and aligning their business process to facilitate a smooth execution. This is further detailed in section 5.7.
- A. 2** Standards compliance: All participants have the required hardware, knowledge and processes to calculate emissions in accordance with the GLEC Framework (2.5). Participants also support the PACT network, which provides the technical data exchange protocol. While PACT standards are followed, the transparent data exchange it defines is not mandatory.
- A. 3** Trusted data sources: All participating organizations are assumed to have access to trusted and certified data sources (e.g., sensors) whose origin and immutability can be verified.
- A. 4** Complete reporting: All parties are expected to report their emissions completely and correctly. This includes the timely submission of activity data and emission results.
- A. 5** Process order: The sequence of business processes is assumed to be correct and strictly followed. This includes that each party completes its process step before the next one begins, and that this order is verifiable through audit mechanisms and receipt chaining.

These requirements and assumptions define the foundation upon which the *TRACE* system was conceptualized, designed, and evaluated. Implications of these design decisions are further discussed in chapter 8.

Chapter 4

Related Work

Carbon Accounting Scientists from various disciplines have already addressed the problem we deal with in the *TRACE* project. Fundamental and practice-oriented requirements and needs originate from research conducted by life cycle experts. Corresponding systems and cryptographic mechanisms that implement verifiability in the context of PCFs originate from specialist literature on computer science.

The work of Jaeger et al. [9] presents the findings of a consortium of life cycle assessment experts on the requirements and challenges of supply chain-wide PCF calculation. According to their findings, supply chain-wide calculation is hampered on the one hand by poor data quality and on the other by a lack of transparency in shared upstream PCFs. In the first case, poor data quality is usually caused by data gaps, e.g. limited availability of life cycle databases, a lack of standardisation of data formats and varying measurement methods with unclear origins. In the second case, companies often have economic incentives not to share their data and calculations transparently. This information is considered sensitive. To resolve the conflict, on the one hand, it must be possible to trace where data comes from and what assumptions are used to perform calculations, and on the other hand, sensitive data must be protected. The authors express the need for a technical platform for the secure and seamless transfer of standardised data that supports a verification mechanism.

One of the most important preliminary studies for our project was provided by Heiss et al. [8] in their paper *Verifiable Carbon Accounting* in Supply Chains. In this paper, the author motivates the need for further development of MRV systems, based on a World Bank report that concludes that the lack of digitalisation and reliance on manual processes makes the determination process prone to errors and causes excessive costs [19]. Furthermore, even digital MRV systems do not sufficiently support product carbon reporting, as they do not enable automated verification processes. Based on this problem assessment, Heiss et al. propose ‘verifiable carbon reporting’ based on zero-knowledge proofs. The system is based on a one-time setup by an auditor and a repeated MRV workflow by the company, its suppliers and the blockchain. During setup, the auditor checks MR schema compliance, creates a zero-knowledge proof program, and the MRV schema for verification, storing these on the blockchain. During the workflow, emissions data is collected, processed confidentially and submitted to the blockchain as a verifiable but non-disclosing zkReport for verification. This ensures authenticity and integrity and enables supply chain partners to automatically verify and share PCF calculations.

Trustworthy Supply Chain Exchange for Product Carbon Footprint explains in detail the problem surrounding the exchange and verification of PCFs [14]. The authors note that systems are needed that enable dynamic PCF calculations. This means that the actual PCF value should be collected based on production data. In addition, established standards allow room for interpretation and do not enable verifiable sharing of PCFs. In practice, certification documents often constitute trade secrets because they contain the bill of materials (BoM): information that is strictly confidential. In the proposed approach, the authors rely on a cloud-based solution in which Verifiable Credentials (VCs) certified by a third party are issued. These are issued after a party in the supply chain has submitted its PCF calculation based on the VCs of upstream suppliers and this has been successfully verified. Using zero-knowledge proofs, holders of PCF VCs can verifiably disclose a selective excerpt of their verifiable certificates. Through the continuous involvement of the third-party certification authority and the selective, provable disclosure of sensitive information, the authors expect PCF reporting to be significantly simplified, even in heterogeneous industries.

The paper "Sustainable Business Practices With Carbon Accounting and Reporting" looks at the problem from a management perspective [12]. The framework proposed by the authors focuses on the use of management practices to control emissions in various areas of the company, such as manufacturing, transport and other domains. The core of the approach is the measurement of CO₂ emissions along critical business processes, particularly in the areas of procurement, operations, logistics and product development. This is intended to provide companies with a means of effectively implementing their strategic environmental goals. Another central component of the framework is the establishment of a data management system that enables automated recording and real-time tracking of emissions. For reporting to stakeholders, the authors recommend following voluntary standards such as the CDP (Carbon Disclosure Project) and the TCFD (Task Force on Climate-related Financial Disclosures). However, these voluntary standards do not adequately address the challenges of complex supply chains, especially with regard to cross-organisational collaboration. According to the authors, the findings, such as the carbon footprint and other KPIs, primarily serve internal and external benchmarking purposes. However, they only contribute to a limited extent to transparency for external stakeholders.

Precompiles In the official *RISC Zero* documentation, the authors state that the current supported precompiles for cryptographic operations and algebraic functions, i.e., SHA-256, RSA, elliptic curve and modular multiplication are susceptible to Timing Attacks. They currently do not provide strict guarantees about constant-time execution and proving time. Thus, they warn the users to be very careful when using any of these precompiles with private data, such as signing a message within the zkVM where an observer can measure the proving time or view cycle counts¹. This leads us to explore the following optimization.

Lazy Signature (Commitment Scheme) In 2024 Swalens et al. [17] present what they call "Lazy Signature" in their paper: "zkStream: a Framework for Trustworthy Stream Processing". Their motivation was to overcome the expensive computation cost of proof generation for signature verification, without the loss of guarantees as confidentiality, provenance, and computational integrity.

¹<https://dev.risczero.com/api/zkvm/precompiles#timing-attacks>

Lazy signature verification outsources the expensive signature verification from the prover to the verifier. They do so by utilizing a commitment scheme, which is a cryptographic primitive that allows one to "commit" to a value (the binding property) while keeping it hidden to others (the hiding property). They use a collision-resistant hash function and if a later operator can regenerate the same hash, it must know the same value (binding them). Since the hash of the value alone is vulnerable to dictionary attacks, adding a secret salt is crucial for confidentiality (hiding) according to [17].

Reducing complexity in the proof generation is extremely relevant considering the scale of transport chains that *TRACE* assumes, thus we implement this optimization in our project.

Chapter 5

System Design

In this chapter, we discuss the system design of the *TRACE* prototype. It is based on our goals described in chapter 3 as well as the application scenario presented in section 5.1. *TRACE* aims to provide a verifiable, trust-minimized and interoperable system that calculates and reports CO₂ emissions at the level of individual transport segments. The focus is the integration of cryptographic verification and proving methods, in particular ZKPs, into a process-oriented system architecture. This is realized through a modular microservice implementation.

The design explicitly follows a distributed approach where individual calculations and program logics are orchestrated in dedicated services. Particularly, the integrity of calculations, the authenticity of input data and extensibility with regard to existing standards such as iLEAP is of importance.

5.1 Scenario

Transport chains are complex in structure and often deviate from the simplified image of linear, sequential peer-to-peer interactions. In practice, standards for emission data exchange like iLEAP define a hierarchical, tree-like structure, in which a Transport Service Provider (TSP) commissions multiple contractors for different transport segments. These contractors only communicate with the TSP instead of directly with peer companies. Figure 5.1 shows a minimal example of such transport chain interactions.

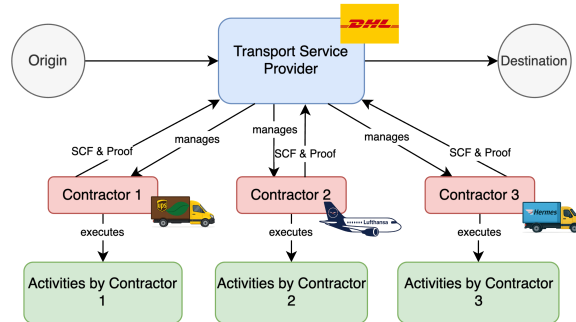


Figure 5.1: Minimal scenario for transport chain hierarchy

This example shows that when a transport is issued, a TSP is requested to find contrac-

tors for each segment of the transport. In this specific case, we have three contractors, one handling transport to an airport, a second handling aerial transportation, and the third completing transport on land. The TSP plans a route and assigns the required contractors with the corresponding segments. Each contractor is responsible for transporting the cargo for a segment of the full journey and also for the carbon emissions caused by it. They perform transport activities as defined in their business processes and after completing their segment, each contractor calculates an aggregated carbon footprint for their services and creates a ZKP over that calculation. Both are sent back to the TSP, which forwards them to the next contractor, who can then incorporate them into their own calculations. The result is one final PCF for the entire transport chain, which could, for example, be made visible as a “DHL Go Green Solutions” label on a parcel.

5.2 Architecture Overview

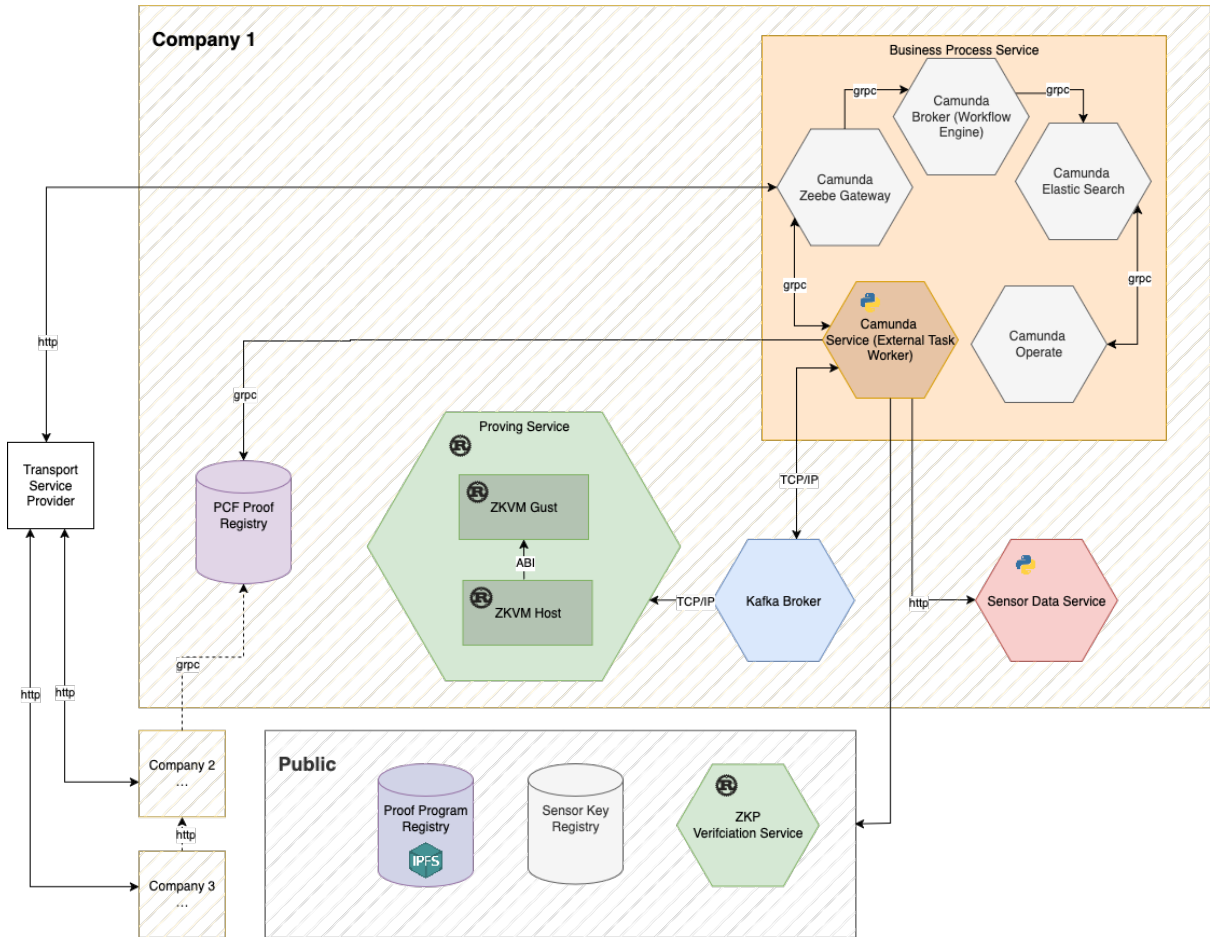


Figure 5.2: System Architecture of *TRACE*

The *TRACE* architecture shown in fig. 5.2 is implemented as a microservice architecture and is designed for operation in a cloud-native environment. It consists of modular, loosely coupled services that use standardized interfaces like REST and gRPC to communicate with each other. Every organization that is part of the transport chain maintains their own instances of private

services of the *TRACE* system and is responsible for calculation of their own emission data and the verification of previous calculations. The architecture is clearly divided into a private area and a public area. While private services are executed within a company, no data or information about their business processes leaves the bounds of that company. The public domain shown in fig. 5.2, on the other hand, consists of central, shared services and registries that exist only once across the system.

By structuring the architecture in this modular way, private as well as public services can be flexibly scaled and expanded with additional registries. The central control of the calculation-relevant processes is carried out via a BPMN based workflow engine (Camunda) which maps the emissions calculations along transport chains and enables the process-centric approach to carbon accounting we aim for. The core of the system is the *Proving Service* that uses the zkVM from *RISC Zero* to perform cryptographically verifiable calculations. The *Sensor Data Service* simulates responses from transport monitoring sensors. Each company's *TRACE* instance also features a *Proof Registry* where emission calculation results and the corresponding artifacts are stored, enabling peer companies to access them. For internal data exchange and the transfer of large amounts of data, a *Kafka* broker is used. The public registries for sensor keys and proof programs are maintained by trusted third parties and provide authenticity guarantees for tools used in calculations, while the verifier service offers verification capabilities independent of the rest of the *TRACE* system.

5.3 Service Components

This chapter describes in detail the function of the service components that were introduced in section 5.2. Each component fulfills a specific functional role within the *TRACE* system, which together enable the calculation, verification, and secure exchange of emission data. By fulfilling these roles, every service contributes to the system goals defined in section 3.1. The following subsections explain the architectural role of each service, its contribution to the overall system, and the system goals it contributes to achieving.

5.3.1 Business Process Service

Architectural Role: Workflow Orchestration and Business Process Integration

Service Contribution: The Business Process Service acts as the central orchestration layer that coordinates carbon footprint calculation workflows across all *TRACE* system components. It manages BPMN-based business processes for PCF tracking, integrates sensor data collection with proof generation services, and ensures seamless data flow between proving, verification, and registry services. The service enables organizations to embed verifiable carbon accounting directly into their existing business process management systems, facilitating automated PCF calculations in compliance with standards such as GLEC, ISO 14083, and PACT/iLEAP, while maintaining data privacy through coordinated zero-knowledge proof workflows.

System Goals Addressed: This service addresses **Goal 1** by enabling all supply chain partners to calculate their PCF through implementing industry-standard carbon accounting methodologies (iLEAP, GLEC, PACT) within business workflows. It fulfills **Goal 4** by controlling data flow and operating within private company infrastructure. The service supports **Goal 3** by integrating verifiable carbon accounting into standard business process management

systems, ensuring adoption within existing business infrastructure rather than requiring parallel systems.

5.3.2 Proving Service

Architectural Role: Cryptographic Verification and Zero-Knowledge Proof Generation

Service Contribution: The Proving Service represents the cryptographic core of *TRACE*, transforming traditional trust-based carbon accounting into mathematically verifiable computations. It receives calculation instructions alongside process data from the Business Process Service, executes them within the zkVM of *RISC Zero*, and produces zero-knowledge proofs that ensure integrity and confidentiality.

System Goals Addressed: This service addresses **Goal 1** and **Goal 2** by providing cryptographic verification of carbon footprint calculations using industry-standard methodologies, ensuring mathematical accuracy and integrity. It fulfills **Goal 4** by generating zero-knowledge proofs that enable verification without revealing sensitive operational data or proprietary calculation methods.

5.3.3 Sensor Data Service

Architectural Role: Authenticated Sensor Integration

Service Contribution: The Sensor Data Service establishes the foundation for trusted carbon footprint calculations by generating authenticated transport measurements. This service simulates the interface to real-world vehicle telematics and IoT sensors, that would capture operational data in production deployments.

In production scenarios, sensors would possess cryptographic keys certified by trusted institutions such as TÜV¹ or equivalent certification bodies. The service would interface with these hardware security modules to generate authenticated measurements that can be traced back to verified sensor origins.

System Goals Addressed: This service addresses **Goal 1** by providing the authenticated operational data required for accurate PCF calculations. It contributes to **Goal 5** by providing digital signatures, which enable the verification of sensor data provenance and tamper-proof integrity through certified hardware security modules.

5.3.4 Sensor Key Registry

Architectural Role: Sensor Key Infrastructure and Sensor Authentication Management

Service Contribution: The Sensor Key Registry functions as the complementary trust infrastructure to the Sensor Data Service, maintaining a registry of authorized public keys and enabling cryptographic verification of sensor authenticity. This service mirrors the operational pattern of established public key infrastructures in industrial IoT deployments.

System Goals Addressed: This service addresses **Goal 5** by maintaining a trusted registry of certified sensor public keys that enables verification of data provenance and integrity. It facilitates **Goal 6** by providing a standardized and audited key registry that enables cross-organizational verification of sensor authenticity across supply chain boundaries.

¹<https://www.tuv.com/germany/>

5.3.5 Verifier Service

Architectural Role: Third-Party Verification Layer

Service Contribution: The Verifier Service operates as an independent cryptographic validation authority that enables third-party verification of carbon emission proofs without requiring access to confidential business data or a specialized zero-knowledge infrastructure. Only access to the ZKPs, which should be verified and do not reveal any information besides the truthfulness of their claims, is needed.

System Goals Addressed: This service addresses **Goal 2** by providing independent mathematical verification of PCF calculations through cryptographic proof validation. The service supports **Goal 8** by allowing external parties to verify carbon footprint claims across organizational boundaries without requiring access to internal systems or data. It contributes to **Goal 6** by replacing traditional audit-based verification with cryptographic proof validation, reducing dependency on trust relationships and manual verification processes.

5.3.6 PCF Registry

Architectural Role: Proof Storage and Inter-Organizational Data Exchange

Service Contribution: The PCF Registry is a self-hosted, access-controlled file storage used to persist the proofs. Access to protected data for external parties is granted solely by the hosting organization. Architecturally, the PCF Registry Service serves as a secure interoperability boundary and professional point of contact between organizations, enabling the exchange of large proofs between companies inside the supply chain.

System Goals Addressed: This service addresses **Goal 8** by providing a secure mechanism for sharing verifiable carbon footprint proofs across organizational boundaries.

5.3.7 Kafka Messaging Infrastructure

Architectural Role: Communication Layer

Service Contribution: zk-STARK receipts typically range from several megabytes to tens of megabytes, with size determined by computational complexity and the number of composed proofs. Traditional RPC mechanisms are inadequate for this use case due to inherent message size limitations - most RPC frameworks impose a 4MB limit by default². This constraint makes standard request-response patterns unsuitable for transferring large proof documents and their generated receipts. Consequently, systems handling large ZKP must implement alternative communication strategies such as streaming protocols. Apache Kafka³ provides the necessary infrastructure through its configurable message size limits and streaming capabilities to handle cryptographic artifacts of varying sizes efficiently.

System Goals Addressed: This infrastructure addresses **Goal 7** by providing high-throughput messaging capabilities that handle large proofs without prohibitive overhead.

²<https://learn.microsoft.com/en-us/aspnet/core/grpc/security?view=aspnetcore-9.0>

³<https://kafka.apache.org/>

5.3.8 Proof Program Registry

Architectural Role: Decentralized Guest Code Storage

Service Contribution: The Proof Program Registry functions as a public repository for guest codes used in ZKP generation. By leveraging decentralized technologies, the registry eliminates the trust and security risks inherent in centralized servers, including concerns about neutrality, reliability, and long-term accessibility. This enables the verification of the program’s logical correctness for each generated ZKP, which in turn guarantees the procedural validity of the calculation (i.e., that the correct sequence of steps was executed). This service combines IPFS for off-chain storage of guest code with blockchain-based smart contracts that store corresponding content hashes. This design ensures that data remains verifiably unaltered and globally accessible while eliminating single points of failure.

System Goals Addressed: This service addresses **Goal 6** by eliminating dependence on centralized authorities through decentralized storage and blockchain verification. The service supports **Goal 8** by providing a neutral, globally accessible registry that enables cross-organizational verification without requiring trust in competitor infrastructure.

5.4 Component Interaction

Figure 5.3 shows the interaction of private components depicted in fig. 5.2 as well as an exemplary call to a public service.

The process centric approach of *TRACE* is clearly visible, since all inter-service requests

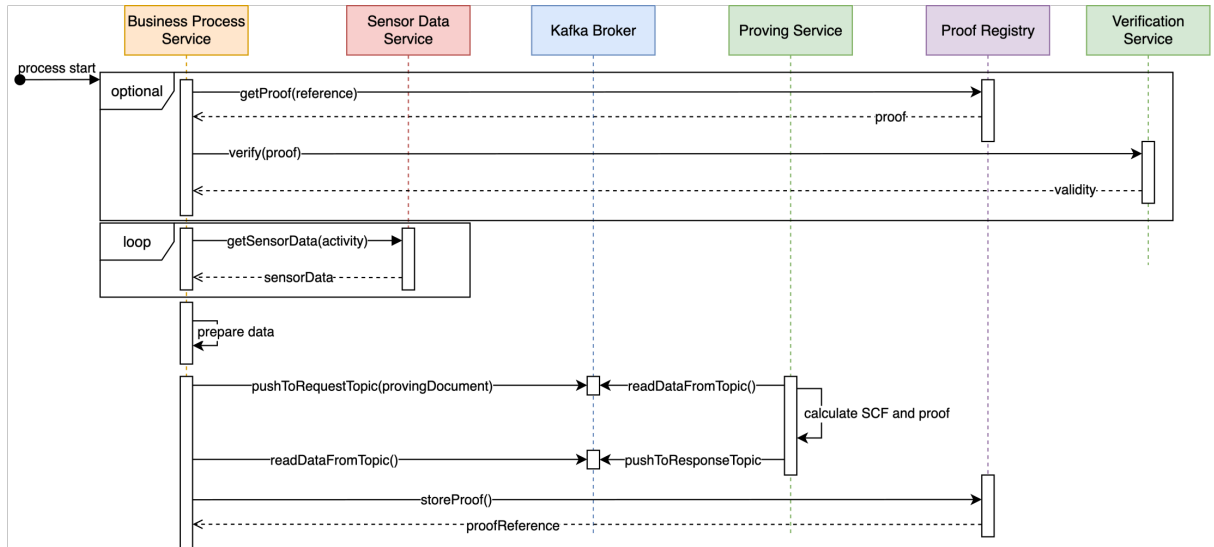


Figure 5.3: Interactions of *TRACE* components

originate from the Business Process Service. It orchestrates the other services and thereby the sequence of service interactions. The sequence starts once a business process executes a transport activity. This entails calling the Sensor Data Service, requesting sensor data from the vehicle corresponding to the executed activity. This is repeated for all transport activities as they arise in the process. Next, additional coefficients required for the later calculation are collected from the process context. Gathered into one compact document, the data created

and received during the business process is stored in a Kafka topic. The Proving Service consumes that data, calculates the PCF for the company's process and creates a proof over that calculation. Proof and calculation result are stored in another Kafka topic, where the Business Process Service consumes it and proceeds to store it in the Proof Registry. In response to that the registry sends a reference to the stored data. The process forwards the reference to the next participant in the transport chain, usually via a TSP, as explained in section 5.1. That company can fetch the proof and the calculation result from the PCF Registry using the reference. With the proof, the public Verification Service is called, enabling independent verification of previous proofs controlled by business processes. That means each company BPM can individually handle the verification of proofs. Not included in the figure is the interaction between companies, refer to fig. 5.1 for that.

5.5 Integrating and extending existing standards

In order to enable interoperable emission reporting across organisational boundaries, we use and build upon existing standards, namely iLEAP and the underlying GLEC, PACT and ISO 14083. Fulfilling Goal 1, these standards define not only how Service Carbon Footprints are calculated but also how data is structured and exchanged in the transportation supply chain context.

Using ZKPs to prove emission calculations can only work with existing standards. In the absence of consensus on calculation methodologies or on the measurement and structuring of underlying data, it becomes exceedingly difficult to assess the correctness of the results. Adhering to standards allows for automation and in our case also enables systematic proving and verification, by also verifying the program logic (section 5.4).

The Product Footprint Template described in section 6.1.1 references the **ProductFootprint** schema⁴, containing metadata for the entire shipment and for the company. This includes a shipment ID that associates data across organizational boundaries to the same shipment. The **extension** field is used to include the iLEAP extension⁵, which we use as defined by the SINE foundation⁶. The accumulated data from each transport activity is structured in the iLEAP TCE schema. This includes a unique ID for the element, a reference to the shipment, the corresponding HOC or TOC element for this activity, as well as the transport distance. The distance adheres to the **GLECDistance** schema, although only the **actual** field is required for the prototype implementation. After calling the Sensor Data Service and receiving a measurement response containing the distance, that field in the TCE is filled. Similarly, HOC and TOC elements for each activity are retrieved from the process, and references are entered into the TCE schema. They contain essential data like the CO₂ intensity values for the category the activity belongs to. Using these standards is mandatory, as it allows for unified logic in the proof generation, based on the assumption that the mentioned fields are present in the proof input data.

The **ProvingDocument** we define is based on **ProductFootprint** and extended with HOC, TOC and signed sensor data. These are appended to the JSON structure of **ProductFootprint**.

⁴<https://wbcsd.github.io/data-model-extensions/spec/#productfootprint>

⁵<https://sine-fdn.github.io/ileap-extension/#introduction>

⁶<https://sine.foundation/>

Each of these elements features a `tceId` field to reference the TCE entry they belong to. This ensures unambiguous de-serialization in the proving service. Alongside the iLEAP extension, we utilize the `extension` field defined by PACT to include our custom `Proof` extension (B.2).

There are fundamental design differences in how data exchanges between companies work in a supply chain, as iLEAP defines, and we implement them. iLEAP assumes transparency as a means to guarantee verifiability in contrast to the ZKP approach we suggest. Therefore, the API endpoints defined for the PACT network can not be provided completely. Implementing `GET /TransportActivityData`⁷ makes no sense as that is the exact data our approach hides. Still, API access to the proof registry of other participants is required and established beforehand, as described in section 5.7. This is a direct substitute for requesting the transport activity data, as a proof of the PCF calculation holds the same value in terms of validating claims on emission reports.

In conclusion, we use the existing standards to ensure agreement on shared data structures and extend them to accommodate for requirements that are unique to our approach, while ensuring to only extend in places that are intended for that purpose.

5.6 The Role of Auditors

As mentioned in chapter 4, current systems rely heavily on auditors to achieve trustworthiness, which introduces significant costs. *TRACE* aims to reduce the requirement for auditors, though it can not avoid them entirely. This section outlines the role of auditors in our design.

The goal is to reduce the use of auditing to individual steps, spot checks or otherwise calculable amounts of auditing that scale dynamically with an increased number of processes.

The Sensor Key Registry is managed by a trusted third-party entity. This is no real drawback, since certifiers like TÜV are required in any solution to limit fraud at the hardware level, and they are relied on in any real-world use case in the industry. The Proof Program Registry works in a similar way, on a software level. It does not encapsulate complex logic, but is an automatable lookup for valid proving logic. Since most of the used programs will be identical, since the underlying calculations are governed by established standards (section 5.5), the auditing overhead is minimal and could even be crowd-sourced, letting the open source community monitor the correctness of registered programs.

Using ZKPs to prove claims on used emissions also simplifies the auditing process, since auditors can widely use "public", or at least non-confidential data, since ZKPs, by design, do not reveal the information they are based on. Rather than continuous monitoring, this approach relies on targeted audits at two key junctures. First, a setup-phase audit validates the artifacts that companies use to create ZKPs, ensuring their transitive validity. Second, a concluding audit verifies the underlying assumption of honest participation, confirming that correct program logic was used and sensor data was not manipulated. In total, this reduces the auditing overhead considerably.

⁷<https://sine-fdn.github.io/ileap-extension/#test-case-004>

5.7 Required Setup and Configuration

The successful deployment and operation of the TRACE system is predicated on the completion of setup and configuration steps. These prerequisites are categorized into organizational, technical, and security domains to ensure a comprehensive and secure foundation for all participating entities.

1. Organizational and Procedural Prerequisites

Adoption of Standards: All participating companies must formally agree to adopt the GLEC Framework for emission calculations and the PACT network protocol for data exchange. This foundational agreement is critical as it ensures that all calculations are consistent and that data is interoperable across organizational boundaries, which is a prerequisite for automated verification.

Business Process Modeling Alignment: Participating companies are required to have their internal business processes formally modeled using BPMN. These existing or newly created models must then be adapted and aligned with *TRACE*'s automated workflow to ensure seamless execution. This alignment involves modifying the process models to incorporate several new, *TRACE*-specific tasks:

- Integration of a verification task at the process initiation to retrieve and validate the cryptographic proof from the preceding participant in the transport chain.
- Addition of service calls to the Sensor Data Service during each transport activity to collect authenticated, signed operational data.
- Inclusion of a concluding sequence of tasks to orchestrate the generation and storage of a new cryptographic proof. This sequence includes invoking the Proving Service to calculate the PCF and generate the ZKP, and subsequently uploading the resulting proof to the organization's private PCF Registry.

Section 6.1.1 discusses an example process with all required tasks.

Resource Verification: Each participating entity is required to possess the necessary in-house knowledge and hardware to comply with the agreed-upon standards. This includes having personnel trained in GHG accounting principles and the required certified sensor hardware for data collection.

2. Data and Communication Preparation

Calculation of HOC/TOC Factors: Companies without existing HOC and TOC emission factors must determine them before participation. These company-specific factors are required for accurate PCF calculations and must be available prior to system operation.

Establishment of Communication Channels: Secure communication channels must be established between all participants to facilitate the exchange of data and cryptographic proof references. This involves the technical configuration of networks and APIs to allow for secure inter-organizational communication between the distributed microservices, thereby allowing, for example, the process engine of a downstream company to retrieve a proof from its upstream counterpart.

Configuration of Access Rights: Access rights for essential data exchange points, particularly the private PCF Registries, must be configured and granted to the relevant partners within the supply chain. As the PCF Registry is a self-hosted, protected service, this step is mandatory to allow subsequent partners to retrieve and verify proofs. This process entails the secure dissemination of authentication credentials (e.g., API keys or access tokens) required to access the registry's API.

3. Auditing Critical Software and Hardware

Audit and Registration of Physical Sensors: A comprehensive, in-person audit of all physical sensors by a trusted third-party auditor is a mandatory prerequisite. The auditor is responsible for conducting a thorough verification process at the company's premises, which includes:

- Confirming that the sensor possesses a valid, certification from an accredited institution, such as TÜV.
- Ensuring the sensor is operating correctly and providing accurate measurements.
- Confirming that the sensor is physically installed and configured in the correct manner and position to guarantee the validity of the collected data.

Only after all these checks are successfully completed will the auditor populate the public Sensor Key Registry with the sensor's public key. This comprehensive audit establishes the root of trust for all subsequent data.

Audit and Registration of Proof Programs: An auditor must validate the software logic (i.e., the ZKP guest programs) used by each company to ensure the calculation logic itself conforms to the agreed upon standards. These verified programs are subsequently registered in the public Proof Program Registry, allowing any verifier to confirm that a given proof was generated using valid, pre-approved logic. Given the generic nature of a program that operates strictly according to standards, it is possible for different companies to use identical programs. The audit process can therefore be significantly reduced if companies use template program logic.

Audit of Business Process Models and Data: External auditors are responsible for checking the accuracy of the modeled processes and the validity of key input data, such as the company-specific HOC and TOC emission factors. The auditor determines whether the model accurately reflects the actual business processes and ensures that all emission-relevant processes

have been considered and equipped with sensors. To ensure ongoing compliance, any subsequent, emission-relevant process changes must be reported to the auditor, and spot-checks may be carried out at arbitrary intervals.

While the validation of HOC and TOC data could eventually be automated using ZKPs, for the scope of this prototype, we assume an auditor fulfills this task.

5.8 Trust System

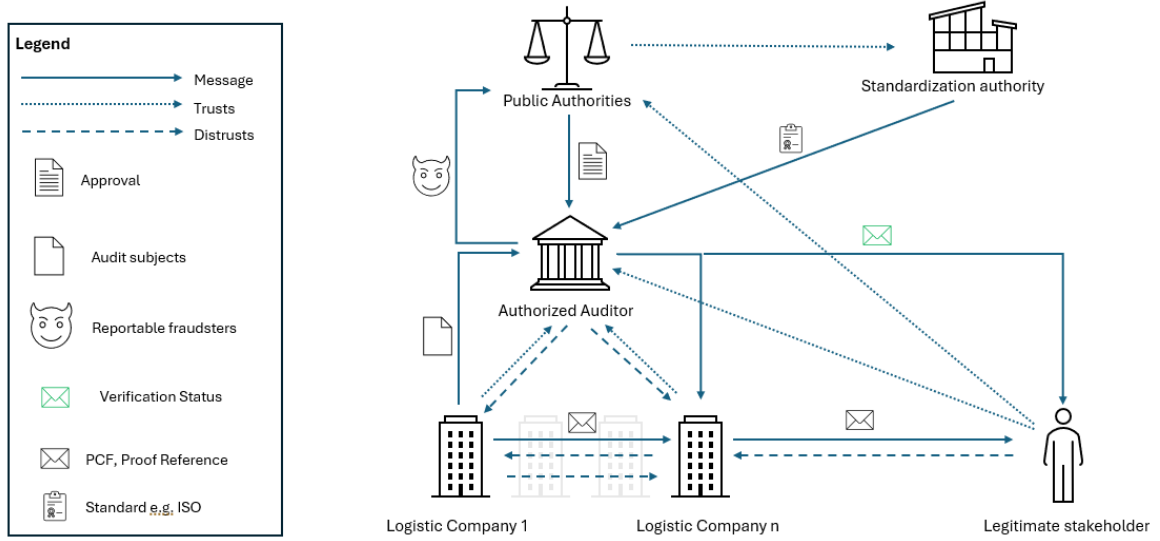


Figure 5.4: Actors and interactions in *TRACE*'s trust system

In order to achieve *TRACE*'s objectives, it is essential to record and explain dependencies with regard to trust systematically. We have opted for a comprehensive presentation that includes two actors who, although they have no influence on the specific system design, nevertheless play a decisive role in determining the framework conditions and shaping the benefits of the system. The interactions of the actors and used artifacts are detailed in fig. 5.4.

The **public authority** is introduced first as the highest authority for creating trust. With its power anchored in the legislative, judicial and executive branches, the state is tasked with creating regulations for emissions accounting and enforcing their implementation, if necessary, through the use of penalties. In its laws, it refers to standards from internationally recognised standardisation organisations, which it officially recognises. In addition, the state maintains authorities that carry out specific monitoring in the transport sector and grant powers, such as the authorisation of auditors. In summary, the public authority thus creates the framework conditions for trustworthy carbon reporting.

The **standardisation authorities** appoint expert committees to develop qualified standardisation projects. These committees often consist of representatives from academia and industry, resulting in standards that are both substantively sound and practically implementable. Standardisation organisations are recognised by the state as national or international standardisation bodies and form the basis for legally binding and internationally consistent procedures.

The **auditor** plays a central role. They are the link for transferring trust from legal provisions to the concrete and correct implementation of these rules. To this end, they receive the audit subjects necessary for building trust from the logistics companies, including information on processes, results of internal audits of the sensor technology used, assumptions for determining emission factors, and the calculation logic of the proof programs used. The auditor treats the information provided by the companies with institutional mistrust, which is overcome through a careful review process. In addition, they operate a verification service that allows audited PCF proofs to be independently verified. In the event of serious violations of the law, the auditor may be obliged to report a so-called "reportable deceiver" to the public authorities.

Logistics companies along the supply chain also mistrust each other. Companies that receive reports fear that they may be embellished to present a better environmental balance sheet. Companies that produce reports, on the other hand, are concerned that sensitive data could be misused when disclosed to business partners. The auditor bridges this gap, as all companies in the supply chain trust him because he acts independently. In the *TRACE* system, companies can therefore share PCF reports and auditor-approved proofs without hesitation and, if necessary, verify them via the Verification Service. At the end of a transport chain is a legitimate stakeholder who must be convinced not only of the emissions value reported by the last link, but also of compliance with all rules throughout the entire supply chain.

The legitimate stakeholder has a profound interest in receiving accurate and traceable information about the emissions generated by logistics companies along the entire supply chain. Such legitimate rights can be either state-guaranteed information rights of consumers or rights of shareholders and business partners. These stakeholders view the reports from the transport chain with fundamental mistrust, but trust in the implementation of legal principles by public authorities. Since the auditor acts on behalf of the public authority, they can also be trusted. In this way, the stakeholder accepts the emissions reports, as the auditor both monitors the process of emissions recording and calculation and provides a fraud-proof verification mechanism.

Chapter 6

Implementation

This chapter highlights implementation details of *TRACE* that contribute to fulfilling the goals listed in chapter 3 on a technical level. The chapter will also explain the more complex components, where conceptual considerations discussed in chapter 5 are not sufficient for a complete understanding. We will concentrate on the business process service and the proving service, with particular emphasis on the performance considerations of the latter. Table 6.1 shows an overview of the languages and tools used for each service, though not all of them will be featured in detail in this chapter. Further information can be found in the public GitHub repository¹.

Table 6.1: Overview of Services, Languages, and Tools

Service	Language	Tools and Libraries
Business Process Service	Python	Camunda, Pyzeebe
Proving Service	Rust	RISC Zero
Verifier Service	Rust	RISC Zero
Sensor Data Service	Python	FastAPI, Cryptography
Sensor Key Registry	Python	FastAPI
PCF Registry	Python	Flask
Proof Program Registry	Go, Rust	IPFS

6.1 Business Process Service

This service embodies the process-centric approach to carbon accounting by managing the execution flow of BPMN process models that represent transport and carbon accounting activities of one company. The service is comprised of multiple microservices, provided by the underlying Camunda Platform 8: The Camunda Zeebe Gateway handles internal and external requests, the Camunda Workflow Engine executes the processes modeled as BPMN files, persistent storage is provided by an Elastic Search instance, and Camunda Operate provides insights into process execution details by monitoring running instances.

Many of the modeled tasks are implemented as external task workers, handling the interaction with other services. Organized in a separate microservice, these workers poll the Camunda

¹<https://github.com/ACP-PCVCF/TRACE>

engine for work items, execute the data collection and document preparation logic, and report completion status back to the process engine, thereby maintaining loose coupling between workflow orchestration and the underlying service implementations. The system is designed to operate within organizations that adopt the *TRACE* framework to extend their existing business processes with verifiable carbon accounting capabilities. Individual process instances execute for the duration required by the transport process, ranging from hours for local delivery operations to weeks for international shipping logistics.

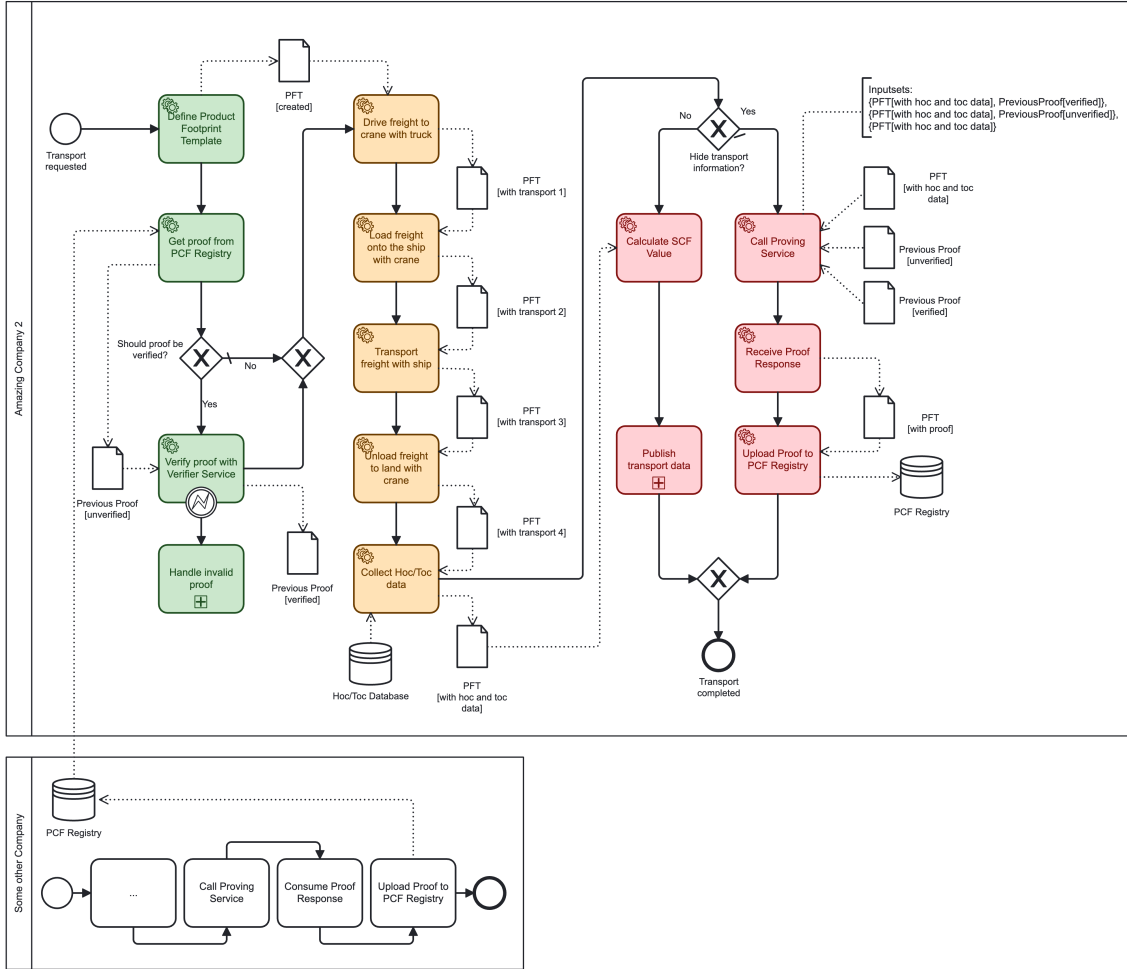


Figure 6.1: Amazing Company 2: Carbon Footprinting Process Workflow

6.1.1 Process Flow Example: Amazing Company 2

To illustrate the system's operation, we examine the BPMN process flow for "Amazing Company 2", the second company operating within a supply chain managed by a TSP. The process demonstrates a complete transportation and carbon footprinting workflow, including actual transportation, confidentiality decisions and composite proof generation. The workflow is structured in three distinct phases: **Setup** (green), **Collect Operational Data** (orange), and **Generate Carbon Footprint** (red), as shown in fig. 6.1.

Phase 1: Setup The process begins when a transport request is initiated. The first phase retrieves and processes the results of the previous company in the transport chain, as well as initializing the data structures needed for the following phases.

Define Product Footprint Template: The system creates a Product Footprint Template (PFT) document (listing 6.1) that serves as the basis for collecting company-specific logistics emissions data throughout the transport process.

Product Footprint Template Structure

```

1 {
2   "productFootprint": {
3     "id": "e4ad7f00-875c-4f20-bbee-d586c3ce62c4",
4     "specVersion": "2.0.0",
5     "version": 0,
6     "created": "2025-07-26T03:40:10.332682",
7     "status": "Active",
8     "companyName": "Amazing Company 2",
9     "companyIds": ["urn:epcids:1fe5f348-9ef2-461a-877b-2c062501edd7"],
10    "productDescription": "Logistics emissions related to shipment with ID SHIP_11777410-3f49-441f-af0a-1c7695a2a029",
11    "productIds": [
12      "urn:pathfinder:product:customcode:vendor-assigned:457a5362-8cc0-4c19-9a19-cdd65006c834"
13    ],
14    "productCategoryCpc": 7290,
15    "productNameCompany": "Shipment with ID SHIP_11777410-3f49-441f-af0a-1c7695a2a029",
16    "pcf": null,
17    "comment": "",
18    "extensions": []
19  }
20 }
```

Listing 6.1: Product Footprint Template Structure

Proof Composition Verification: Since the company is not the first in the transport chain, it automatically receives a proof reference from the TSP and retrieves that proof from the corresponding companies' PCF Registry through the "Get proof from PCF Registry" activity. The process then reaches a decision gateway to determine if an early proof verification is required for business logic. When verification is needed, the system validates the previous proof through the public Verifier Service. If validation fails, the process handles the invalid proof through appropriate error handling mechanisms, defined by the company or the transport chain. This verification is always additional to the one performed while creating a new valid proof during Phase 3.

Phase 2: Collect Operational Data The second phase encompasses the systematic collection of operational transport data from all logistics operations required for the transport process. Each operation extends the single PFT document created in Phase 1.

For each transport operation, the system interfaces with the sensor service to gather real-world operational parameters and appends this data, signed by the sensor, to the continuously evolving PFT document. This results in one TCE element for each transport activity. At the end of Phase 2, the PFT document contains activity data for all emission-creating activities. Some fields in the TCE schema definition, detailed in section 5.5, need to be related to TOC or HOC data. The final step of phase 2, therefore, is to collect these predetermined coefficients from a company-specific database and append that information to the PFT. This document now

contains all input data required to calculate a PCF. Collecting and structuring the data in this way allows for easier data transfer between services and storing the data for potential internal analysis.

Phase 3: Generate Carbon Footprint The final phase handles the generation and storage of carbon footprint proofs based on organizational privacy requirements. A decision gateway determines whether transport information should be hidden or disclosed.

Privacy Path: When organizations select data privacy, the system follows the zero-knowledge proof pathway:

1. **Call Proving Service:** The system calls the proving service with all the collected data, including the previous proof, for PCF calculation and ZKP generation. B.1 shows an example for such a `ProvingDocument`.
2. **Receive Proof Response:** The response from the Proving Service is awaited. It contains the PCF for the company's process and a cryptographic proof over that calculation that enables verification without revealing sensitive operational parameters.
3. **Upload Proof to PCF Registry:** The generated proof is stored in the company's PCF Registry for future reference and proof continuity.

Transparency Path: When organizations choose transparency, the system follows the direct calculation pathway:

1. **Calculate PCF Value:** The system directly computes the PCF value using the collected operational data.
2. **Publish Transport Data:** All sensitive operational input data required for PCF calculation is disclosed to maintain trust and verifiability within the supply chain network.

This approach ensures that each organization can verify the computational integrity of previous steps in the supply chain while maintaining the flexibility to choose between privacy-preserving zero-knowledge proofs or transparent disclosure based on their organizational requirements. The process orchestration seamlessly integrates traditional business process management with cryptographic verification, enabling trustworthy carbon accounting across organizational boundaries.

TRACE does not prescribe specific actions when a previous proof fails verification, nor does it mandate disclosure requirements when organizations choose not to utilize the proving service. These decisions remain within the purview of individual companies and their supply chain partners. It is the responsibility of participating organizations and the broader supply chain network to establish common standards regarding data transparency requirements and to develop appropriate protocols for addressing non-compliance or misbehavior from chain participants.

6.2 PCF Calculation and Proof Generation

The most critical task in the implementation of the calculation and proving system is to achieve a good performance across the entire transport chain, since zk-STARKs are notoriously hard

to compute, making the proving service the supposed bottleneck by design. To fulfill Goal 7, we carefully crafted an implementation that enables improved performance while keeping all private data confidential (Goal 4) and upholding the known security assumptions of *RISC Zero*.

6.2.1 Sensor Data Signatures

All sensor data used in the PCF calculation is signed (PKCS1v15) and later verified to fulfill Goal 5, ensuring that data is not manipulated and originates from a trusted source. Verifying signatures is a computationally intensive yet indispensable operation, leading us to adopt the lazy signature scheme described in chapter 4. For each message the sensor data service sends, it generates a random salt, concatenates the value with it, hashes it and signs the hash. This first step enables optimizations during the proof generation (section 6.2.2).

6.2.2 Proving Service

The Proving Service represents the cryptographic core of *TRACE*. It transforms the comprehensive **ProvingDocument** generated by the Business Process Service into a cryptographically verifiable zero-knowledge proof and produces a **ProductProof** response (B.2). Figure 6.2 outlines the entire sequence of interactions that is executed in the implementation of the proving service. This section covers the depicted implementation details, while the diagram offers orientation on where in the process an explanation can be placed.

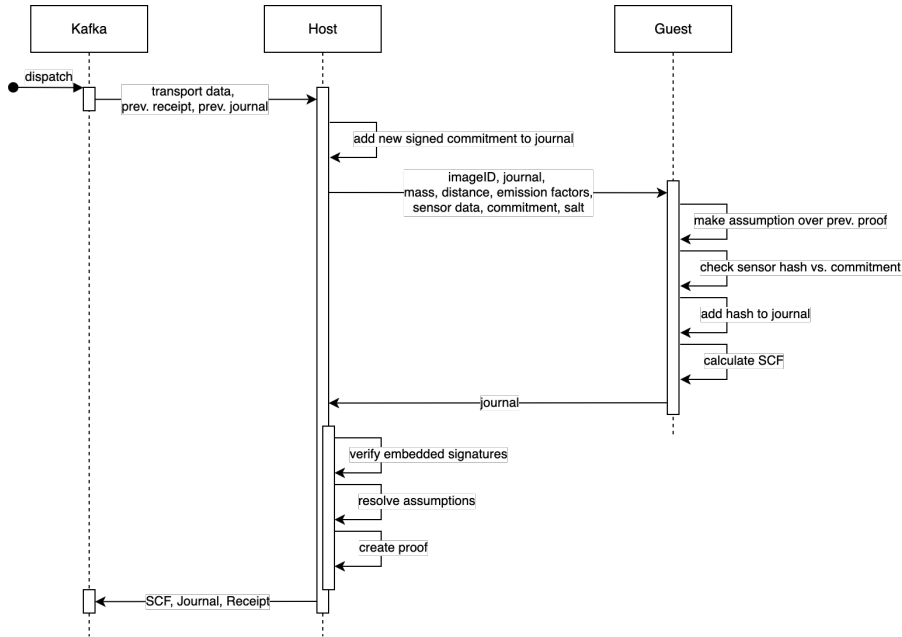


Figure 6.2: Proving Process with involved components and required data

Proving Process

The host program manages the execution environment and coordinates the interaction between external data sources and the zero-knowledge virtual machine. The sensor data, including signatures, salt and commitment as described in section 6.2 are part of the **ProvingDocument**.

The host establishes the proving environment for the guest, providing the `ProvingDocument` as input data.

Signature verification Following the lazy signature scheme, and as displayed in fig. 6.2, the host adds signature and commitment to the public journal for the following participant in the transport chain to verify them. It now contains the signatures and commitments of all previous and current sensor measurements.

The guest begins by validating the input data. It recomputes the commitment from the raw data and salt, and upon successful validation against the provided commitment, writes the result to the public journal. This action attests that subsequent calculations rely on data corresponding to the verified commitment. Following this step, the guest calculates the PCF value in accordance with the GLEC framework using inputs from the `ProvingDocument`, and appends this result to the journal as well.

Once the guest has completed the calculation in its isolated environment, the host verifies the signatures from the previous company’s sensor measurements, which will be done for the current measurements by the following company. The host extracts the hash commitment created in the previous companies guest method. By verifying the proof, as described in 6.2.2, it is ensured that the program was executed correctly for the given public inputs and outputs, thus linking the confidential sensor messages to their public hashes.

Proof Composition Proof composition in the *RISC Zero* zkVM enables the verification of existing proofs within new proof computations, which is considered the gold standard for scalable zero-knowledge applications. Figure 6.3 illustrates this in a simplified manner. The system operates through an assumption-resolution mechanism rather than direct proof verification within the guest program. The core mechanism relies on the `env::verify()` function, which performs cryptographic verification of previous proofs without executing their full computation within the current zkVM instance.

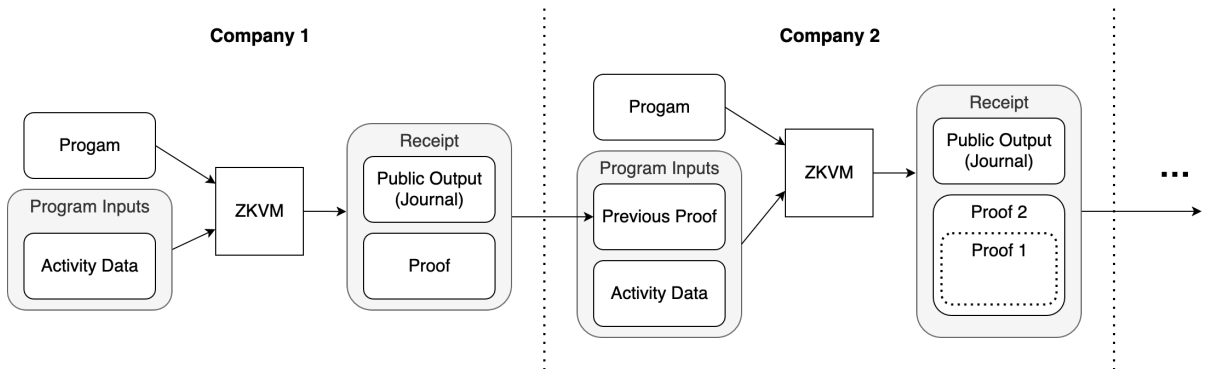


Figure 6.3: Proof Composition in a Transport Chain, adopted from <https://risczero.com/blog/proof-composition>

When `env::verify(image_id, journal_bytes)` is invoked within the guest program, the zkVM performs the following operations:

1. **Cryptographic Verification:** The function verifies that the provided journal was indeed produced by executing the program identified by `image_id`. This verification process

validates the cryptographic integrity of the previous proof without re-executing the underlying computation, ensuring that the journal contents are authentic and have not been tampered with.

2. **Assumption Addition:** Rather than performing full proof verification (which would be computationally expensive), the zkVM adds an assumption to the current `ReceiptClaim`. This assumption states: "The provided journal is valid, assuming the corresponding proof receipt is genuine." This creates a "conditional receipt" that depends on the validity of the referenced proof. This assumption-resolution approach is what makes composition computationally efficient compared to full recursive verification, which would require re-executing all previous computations within the guest.
3. **Journal Content Extraction:** Upon successful verification, the function enables the guest program to decode and extract the computational results (in our case, the PCF values) from the verified journal. This allows the current computation to build upon previous results without accessing the underlying sensitive data that produced those results.

The assumption-resolution mechanism in *RISC Zero* transforms conditional receipts into unconditional proofs. When a prover generates a receipt, the system's optimized recursion circuits automatically verify all corresponding proof receipts, thereby resolving any outstanding assumptions. The efficiency of this process is critical, enabling the practical and economically viable deployment of applications that require numerous composite proofs.

Proving System Overview

For an even more in-depth explanation of the guest and host functionality, we provide pseudo code for both (B.3).

The proof composition feature elevates *TRACE* from a single-party tool into a comprehensive, multi-party supply chain solution, enabling a powerful combination of transparency and confidentiality for carbon accounting across organizational boundaries. Each composite proof encapsulates the entire chain of verified computations, supporting efforts to determine the completeness of the proven data in the chain. Furthermore, this design significantly improves economic viability by offloading two computationally expensive operations—proof verification and signature verification (via the lazy signature scheme)—to the less resource-intensive host environment.

The implementation of the PCF calculation and proof generation facilitates the fulfillment of Goal 4, Goal 7 and Goal 8, which is further explored in chapter 8.

6.3 Deployment

TRACE uses containerized microservices for flexible deployment across different environments. Each service includes a Dockerfile, Helm charts and Kubernetes manifests for deployment flexibility. Container images are automatically built through GitHub Actions and published to GitHub Container Registry. Deployment is managed via Helm charts and Kubernetes manifests, which are adaptable for both large-scale cloud platforms (Amazon EKS, Google GKE,

Azure AKS) and local development clusters like kind or minikube. By pulling pre-built images directly from the registry, developers can avoid time-consuming local builds, significantly speeding up both initial setup and ongoing development.

6.4 Runtime Operation

TRACE is designed to operate in a fully automated manner. Once the initial setup is completed, all operational prerequisites are satisfied (section 5.7), and the required services are deployed across participating organizations, the system can, in principle, function without human intervention. The degree of automation, however, ultimately depends on the specific business processes of each company. Activities that inherently require human judgment or decision-making will continue to necessitate manual involvement when executed within *TRACE*, whereas all *TRACE*-specific tasks are executed automatically.

Chapter 7

Benchmarking & Methodology

TRACE is a composite of several interconnected services, each contributing to the overall performance. However, preliminary tests consistently showed that the proving service is the single most critical component in terms of resource consumption and execution time.

Given the overwhelming dominance, we focus our benchmarking efforts exclusively on the proving service. This approach allows not only for a thorough analysis of the most significant performance bottleneck, but also enables easy variation of the underlying transport chains without the need to create extensive BPMN models for our Business Process Service as an orchestrator.

7.1 Metrics

To effectively evaluate the performance of our proving service and to compare different approaches proving the whole transport chain, we measure several key metrics. These metrics provide a comprehensive view of the program’s efficiency, resource consumption and proof size.

- **Proving Time (Seconds):** This metric measures the total time required for the proving service to generate a verifiable proof for the execution of the guest program. It is a crucial indicator of the practical performance and scalability of the program, directly impacting user experience and system throughput. Proving time can be influenced by the complexity of the computational logic, which includes the complexity of the verification of previous proofs and the underlying hardware.
- **Input Size (Bytes):** The input size represents the amount of data fed into the host program. This differs from the actual data fed directly into the guest program, since previous proofs are not part of the direct guest input, but it is crucial to generate a verifiable proof. This metric is affected by the size of the previous proofs and the complexity of the underlying transport chain.
- **Proof Size (Bytes):** The proof size, or host program’s output size, quantifies the size of the resulting proof including any public outputs. This is also impacted by the size of previous proofs and the complexity of the underlying transport chain.
- **RISC-V Cycles (Cycles):** This metric represents the total number of RISC-V instructions executed by the program logic within the *RISC Zero* zkVM. It’s a low-level,

hardware-agnostic measure of the computational work performed. It directly correlates with the computational cost of the program. These total cycles can also be subdivided into different categories, to better compare different implementations of the same algorithm, though we only focus on the comparison of different proving strategies. This metric is impacted by the complexity of the execution logic and the input of the guest program, therefore also impacted by the underlying transport chain and previous proofs.

7.2 Benchmarking setup

In order to achieve meaningful results, the measurements of the proving service are carried out under controlled conditions to minimize external influences as far as possible. The tests are performed on a standard system that is not used for any other purposes during the measurements. The setup includes the specific hardware and software components as well as the methodology used to collect the data.

7.2.1 Hardware specifications

- **Processor (CPU):** Intel(R) Core(TM) i5-3570K CPU @ 3.40Ghz
- **Memory (RAM):** 16GB DDR3 1600Mhz
- **Storage medium:** NVMe SSD 520MB/s

7.2.2 Software specifications

- **Operation System (OS):** Debian 12.11
- **RISC Zero SDK:** Version 2.0.2
- **RISC Zero zkVM:** Version 2.0.2
- **Rust Compiler:** rustc 1.85.0

7.3 Experiments and Rationale

The primary goal of our experiments is to investigate the applicability and performance of our system, specifically focusing on how the proving service handles different proving strategies. We measure the defined metrics – proving time and input/output size – to evaluate not only our implementation but also the systems efficiency and scalability under various scenarios.

7.3.1 Methodology

To ensure reproducibility of the experiments and comparability between different approaches, we first generate 20 different proving documents, with a randomized number n , $n \in \mathbb{N}$ with $1 \leq n \leq 4$, of TCE with type TOC, including sensor data with hash, key and signature, and a randomized number m , $m \in \mathbb{N}$ with $0 \leq m \leq n$, of TCE with type HOC. This allows us to include variation in the underlying transport chain, therefore, to analyze real world applicability, but also to prevent outliers to ensure comparability between different approaches. For the sake of completeness and for a deeper analysis, the generated sets of values are listed in table 7.1.

1: (4,2)	2: (2,1)	3: (1,1)	4: (4,2)
5: (1,0)	6: (1,1)	7: (4,2)	8: (2,0)
9: (1,0)	10: (4,4)	11: (4,3)	12: (2,0)
13: (4,0)	14: (2,0)	15: (1,0)	16: (3,1)
17: (2,2)	18: (1,1)	19: (1,1)	20: (3,2)

Table 7.1: Number of TOC and HOC element in benchmark input data, index: (n, m)

Other parameters of the proving documents, such as descriptions, names, and others, are not randomized as they do not influence the actual resource-heavy computation.

All experiments are conducted independently of each other. No cool-down phase is required between the individual runs, as the zkVM is reinitialized for each proving step, ensuring a clean state for every measurement.

Due to the long runtime per individual proving step and also per experiment, short-term fluctuations in system utilization are averaged by background processes. The results therefore represent average performance over this period. Nevertheless, the possibility of influence from unpredictable, long-term background processes on the standard Debian system must be mentioned as a limitation of the setup.

7.3.2 Experiment

Our rationale for this experiment is to compare the performance of proof composition against alternative methods, providing a comprehensive understanding of the trade-offs involved. We design three distinct experimental setups:

1. **Proof Composition Chain:** In the first experiment, we investigate a proof composition chain. The proving service is executed 20 times with the generated proving documents, with each subsequent run taking the proof of the previous run as part of its current proving document. This sequential process creates a single, verifiable proof chain and allows us to measure the individual and cumulative time and the individual input and proof size, and most importantly the final size for a fully composed proof.
2. **Individual and Aggregated Proofs:** The second experiment involves a two-step process to serve as a baseline for the composition chain and a comparison for two different approaches. First, we generate 20 individual proofs for each of the 20 proving documents separately. This gives us a clear picture of the performance of a single proving run without the overhead of composition. Second, we create a single aggregated proof over all 20 of these individual proofs. This step measures the efficiency of generating a single final proof from a set of non-composed proofs. This scenario reflects two possible different solutions to our main approach where contractors create independent proofs, therefore not affected by previous participants of the transport chain, while retaining the same confidentiality.
3. **Single Proof:** For our final experiment, we create a single proof over one proving document with the TCEs of all 20 generated proving documents. This approach reflects a scenario where a TSP gets all the transport information data from their contractors in pure form. However, this approach does not offer the same confidentiality as the others.

Chapter 8

Evaluation

In this chapter we conduct the evaluation for the benchmark described in section 7.3 as well as how well the initially set requirements are fulfilled.

8.1 Proof Generation Performance

The primary purpose of conducting the benchmarks is to evaluate performance optimization efforts. ZKPs, especially zk-STARKs require a considerable amount of computational power, thus exhausting optimization potential is necessary in respect to real world transport scenarios that involve many participant, each with a considerable number of steps in their processes and an overall great number of products that are transported daily, worldwide.

Figure 8.1 shows the time required to create 20 consecutive proofs in the different strategies described in section 7.3. Each of the bars has indicators for intermediate results – each line representing the time it took to create a proof, starting with the first at the bottom. Creating nested proofs takes the longest, though creating sequential proofs and aggregating them is not considerably faster, while providing weaker assurances to the validity of each proof until the aggregating proof is completed. The third bar demonstrates that despite the optimizations to the proof composition and verification (section 6.2.2), proofs can be generated much faster without these steps. Because verifying every proof is indispensable to fulfill computational integrity, the aggregating proof can not be omitted, resulting in similar overall proving times for both approaches. Performing the calculation and proof generation on aggregated input data of all companies results in one third of the proving time of the other approaches. Is is only relevant as a reference for the overhead needed to achieve verifiability and confidentiality (Goal 4) for each individual company, since this approach can only be achieved by using a shared trusted entity to perform the calculation and proof generation, which would not hold any confidentiality guarantee and requires additional trust assumptions.

The impact of that overhead grows linearly in size and plateaus in terms of time as shown in fig. 8.2. Since we have established, that the aggregating approach offers no measurable advantage over the nesting approach we only evaluate the latter one further.

The proving time is directly proportional to the complexity of the calculation, measured in RISC-V cycles during the execution of the guest program, as depicted in fig. 8.1. These cycles can be subdivided into user-cycles, the actual computation of the guest program, paging-cycles, for handling memory access, and reserved cycles. *RISC Zero* always reserves a fixed number

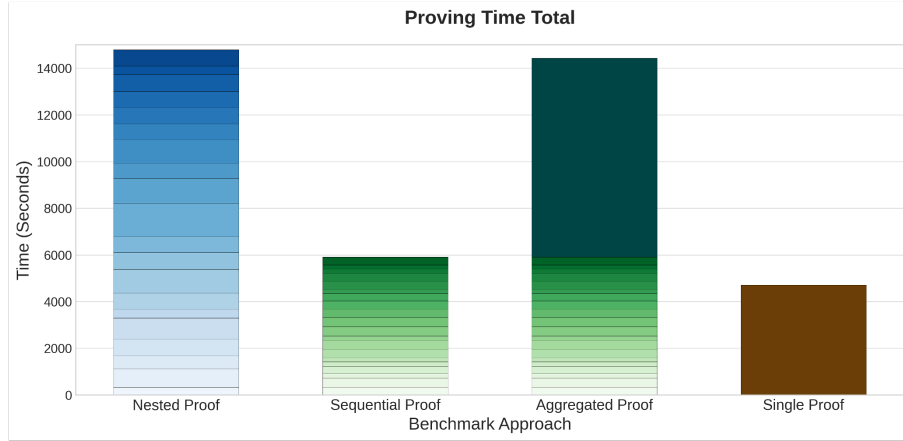


Figure 8.1: Proving Time across different proof chaining strategies

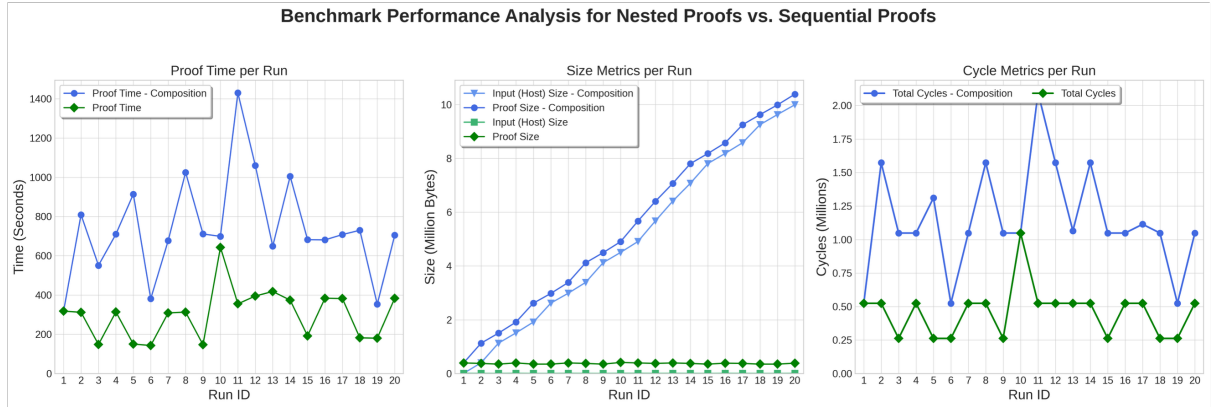


Figure 8.2: Performance Comparison for Nested and Sequential Proofs

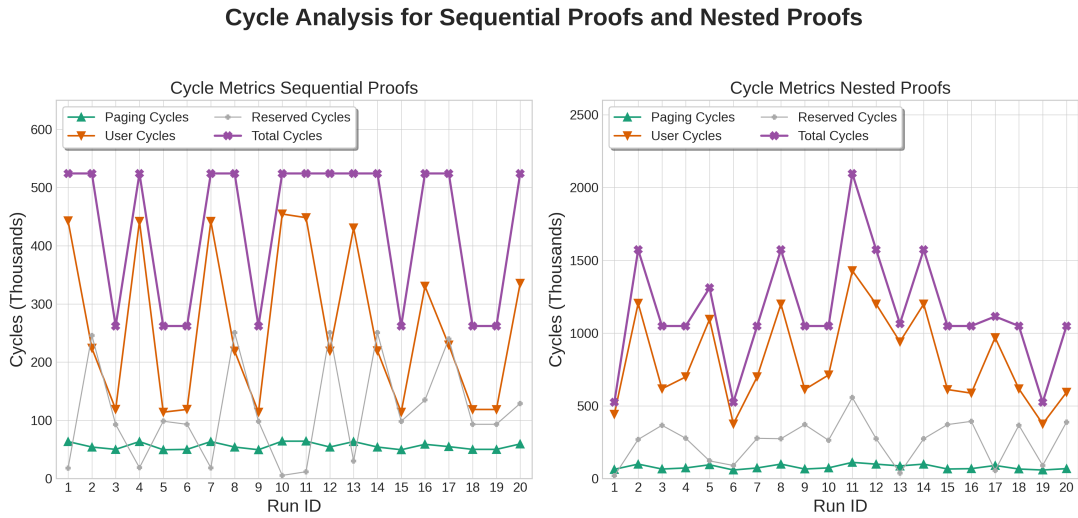


Figure 8.3: Cycle Analysis for Sequential and Nested Proofs

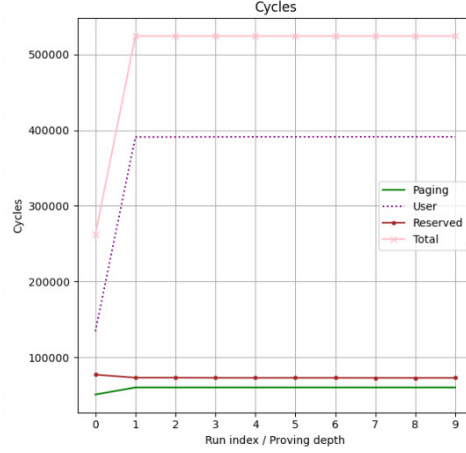


Figure 8.4: Proving Cycles for 10 Nested Proofs over Identical Input Data

of Risc-V cycles for execution, which can also be seen in the raw data ^{1 2}. If the sum of user-cycles and paging-cycles reaches a certain amount, the total-cycles will be increased by a fixed amount; the difference are reserved-cycles. That is why we see discrete values for the total-cycles in fig. 8.2 and fig. 8.3. When we compare the sequential proofs in fig. 8.3 with table 7.1 we can see the actual impact of the number of TCE elements with TOC and HOC. A large part of the computing effort of the guest program is incurred when hashing the sensor commitments, which occurs for each TCE element of type TOC. Thus, there is a strong correlation between the number of these TCE elements and the number of user-cycles. The number of TOC is between 1 and 4 in our benchmark, which is why user-cycles per proof can be found on roughly one of four horizontal lines in fig. 8.3. The spikes and slopes of the graph for the user-cycles for nested proofs in fig. 8.3 are shifted by one between approaches, which means: When a proof is computationally complex, the following proof, nesting this one, will be more complex as well. This also means that despite the nesting and the linearly growing size of the proof, the cycles and the proving time do not grow at all with increasing depth of nesting. All variation in the y-axis values in the plots can be attributed to the differences in the input data. This is also visible in fig. 8.4, where nested proofs are created from identical input data. Notably, but not surprisingly, in fig. 8.1 the cycles and time are identical between the two approaches for the first proof, which does not contain a previous proof.

Overall the nested approach scales very well with increasing participants in the chain, but exhibits considerable variance caused by specific input parameters, like the number of TOC elements, while others (like HOC elements) have no noticeable inference.

The benchmark results approve our nested proving approach as viable, considering the scaling with longer chains and similar proving performance to other approaches that fulfill comparable trust and confidentiality requirements. For the default use case we envision for proof chaining, the nesting approach is suited better than the aggregation approach. In cases, where a single contractor can not muster the computational power or bare the time to create

¹https://github.com/ACP-PCVCF/proving-service/blob/main/benchmarks/bench_composition_0.csv

²https://github.com/ACP-PCVCF/proving-service/blob/main/benchmarks/bench_proofaggregation_0.csv

a nested proof, this is still a valid option, though it requires small changes in the process to accommodate for the flexibility of every company choosing to create an individual proof instead of a nested one. A possible scenario would be, that all contractors with the same TSP create individual proofs and the TSP creates an aggregating proof over that. This shifts the chaining responsibility and computational overhead to the TSP, lowering the starting hurdle for companies with less resources. A considerable drawback is, that the trust no lies entirely with the TSPs, meaning it takes a longer chain, with more TSPs to guarantee enough separate instances that can verify each others claims.

8.2 Trust assumptions lazy signature verification

This section analyzes the security properties of our proposed signature creation and verification framework, demonstrating how it upholds confidentiality, provenance, and computational integrity through the integration of lazy signatures.

The lazy signature framework, as defined by Swalens et al. [17], shifts the computational burden of signature verification from the prover to the verifier. In a zero-trust environment where the entities operating verifier nodes are considered untrusted, this trust assumption is instead placed upon a neutral, external auditor (e.g., TÜV). This auditor takes a more important role in the system design. The balance between computational performance (item Goal 7) and minimizing auditing overhead (item Goal 6) is maintained. The following threat model analysis illustrates the auditor’s critical role in mitigating potential attacks.

Complete Input Forgery (AS 1)

Threat: An adversarial host, operated by an untrusted entity, replaces the entire set of inputs – the salt, message, signature, and public key – with a new, self-generated set. The host also computes a valid new commit from its forged message and salt. The guest’s internal hash comparison would pass, leading it to record the fraudulent data in its journal.

Mitigation: The auditor validates the signature using the provided public key, but crucially, it also verifies that this public key is listed in a trusted and certified Sensor Key Registry. Since the adversary does not possess the private key corresponding to any registered public key, it cannot produce a valid signature for a trusted key. The auditor’s check against the registry thus invalidates the fraudulent entry, as shown in fig. 8.5.. This check can also be performed in an automated way by the following company in the supply chain, without an auditor, given the audited Sensor Key Registry.

Partial Input Forgery (AS 2)

Threat: The host replaces the original message with a fraudulent one, generates a new salt, and computes a new commit based on these forged inputs. It presents the original, valid signature and public key alongside the new, forged commit. As in 8.2, the guest’s internal hash validation passes, and it records the manipulated data.

Mitigation: The auditor detects this forgery by verifying the original signature against the commit provided in the journal. The signature was created for the original commit, not the fraudulent one supplied by the host. Consequently, the signature verification fails, and the

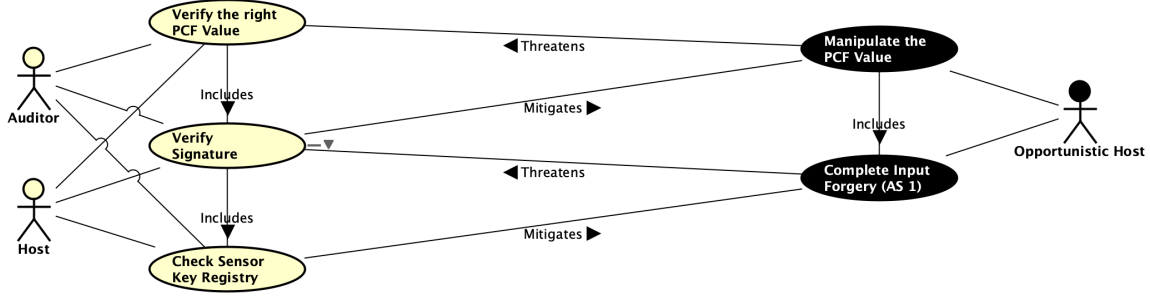


Figure 8.5: AS 1. Use/misuse-case diagram. Use-case elements appear on the left, the misuse case elements on the right. Leaned on [1]

manipulation is exposed, as depicted in fig. 8.6. This check is also performed in an automated way by the following company in the transport chain, as described in section 6.2.2. However, a trusted third party has a stronger argument for the truthfulness of fraud allegations than a peer company.

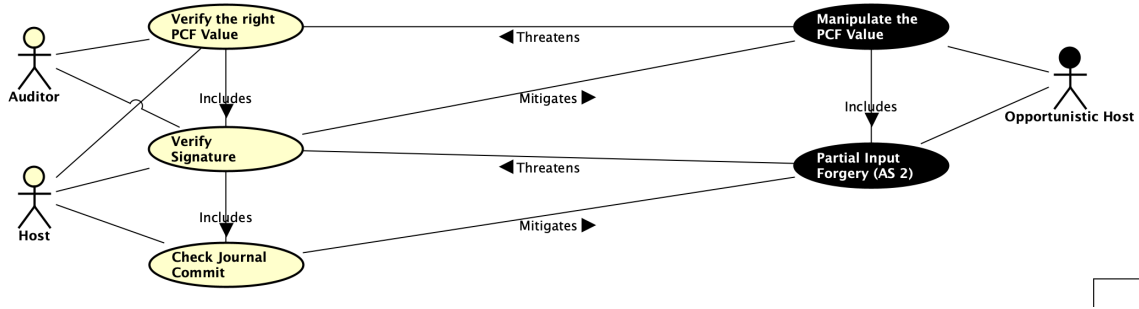


Figure 8.6: AS 2. Use/misuse-case diagram. Use-case elements appear on the left, the misuse case elements on the right. Leaned on [1]

Nested Proofs: Public Value (PCF) Manipulation (AS 3)

Threat: In a chain of nested proofs, a host correctly verifies a previous proof but then attempts to use a different PCF for subsequent computations, deviating from the authentic value recorded in the previous journal entry.

Mitigation: The entire sequence (verification of the previous proof, extraction of its public output, and the subsequent computation) is executed within the attested environment of the current guest. The integrity of this entire computational logic is itself proven. The auditor verifies this proof by checking the hash of the executed program against a trusted Proof Program Registry. Any deviation from the registered, correct program logic results in a verification failure, thereby detecting the manipulation, as illustrated in fig. 8.7. This check can also be performed in an automated way by the following company in the transport chain. However, a trusted third party has a stronger argument for the truthfulness of fraud allegations than a peer company.

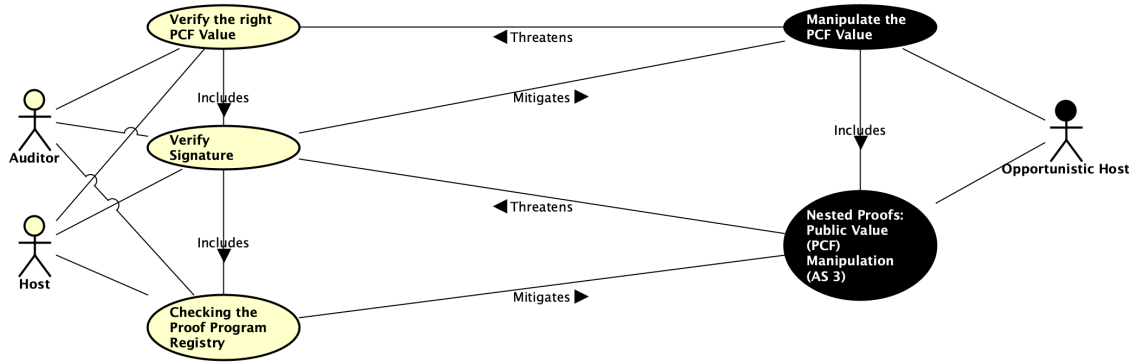


Figure 8.7: AS 3. Use/misuse-case diagram. Use-case elements appear on the left, the misuse case elements on the right. Leaned on [1]

Nested Proofs: Forged Proof Substitution (AS 4)

Threat: A malicious host attempts to substitute the genuine previous proof with a different, forged proof, such as one created in the attacks described in section 8.2 or section 8.2. The host then instructs the guest to proceed with the public value from this substituted proof’s journal.

Mitigation: The protocol requires that critical data from the ”inner” (previous) proof’s journal is explicitly carried forward and made visible in the ”outer” (current) proof’s public journal. This allows the auditor to apply the same verification checks described in AS 1 and AS 2 to the inner journal’s data. Any forgery within the substituted proof is therefore detected, a process outlined in the diagram in fig. 8.8. This check can also be performed in an automated way by the following company in the transport chain, since all required data is publicly visible in the journal. However, a trusted third party has a stronger argument for the truthfulness of fraud allegations than a peer company.

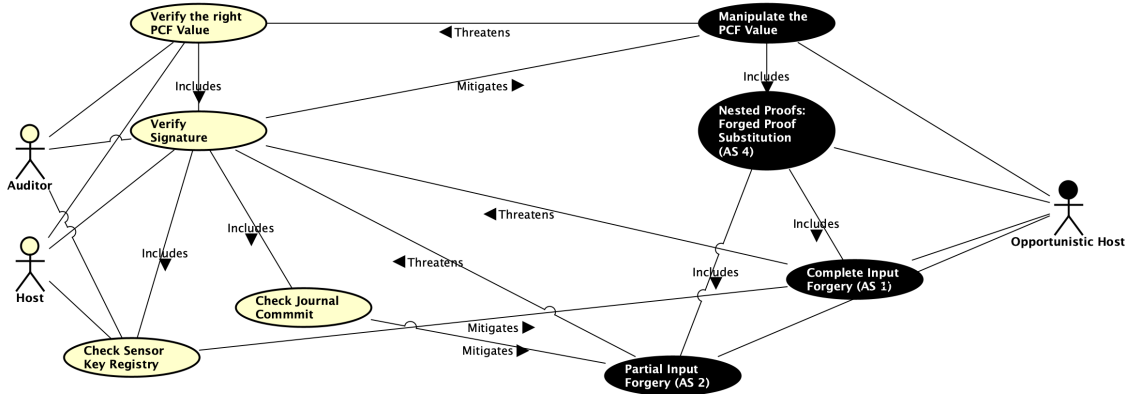


Figure 8.8: AS 4. Use/misuse-case diagram. Use-case elements appear on the left, the misuse case elements on the right. Leaned on [1]

Nested Proofs: Proof Replay Attack (AS 5)

Threat: The host substitutes the correct previous proof with a different, older, but otherwise valid proof from the same chain. This constitutes a replay or "double-spending" attack, where a past state is presented as the current one.

Mitigation: This attack is primarily countered by two mechanisms. First, the inclusion of a high-entropy random salt in each commitment makes the accidental or malicious generation of a duplicate commit cryptographically infeasible. Second, to provide a deterministic guarantee, the auditor verifies each new commit against an append-only, publicly accessible ledger of all historical commits. The public nature of this ledger poses no confidentiality risk, as it contains only commit hashes, not the underlying private data. Any attempt to replay a commit is immediately identified as a duplicate, and the proof is rejected, modeled in fig. 8.8. This ledger could be maintained and synchronized via a smart contract on a distributed ledger for enhanced integrity. This check can also be performed in an automated way by the following company in the transport chain. However, a trusted third party has a stronger argument for the truthfulness of fraud allegations than a peer company. The current system design does not feature the maintenance of the described ledger. Due to the modular design, this extension is not an issue.

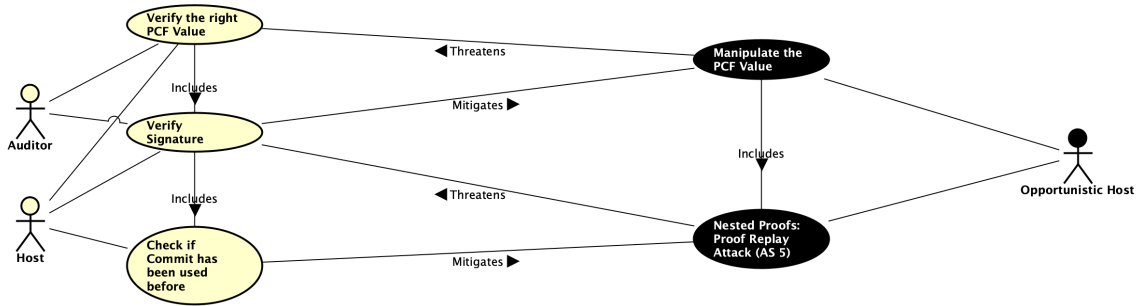


Figure 8.9: AS 5. Use/misuse-case diagram. Use-case elements appear on the left, the misuse case elements on the right. Leaned on [1]

Conclusion of trust assumptions towards lazy signatures

The threat analysis and the general Use/misuse diagram in fig. B.1 demonstrates that the lazy signature protocol, when augmented with the specified auditing measures, upholds the security guarantees outlined by Swalens et al. [17]:

- *Confidentiality* is preserved because the sensitive data is hidden within a cryptographic commitment, never exposed to the untrusted host or verifier.
- *Provenance* is guaranteed because the auditor verifies the signature against a public key authenticated by the trusted Sensor Key Registry, ensuring the data originates from a known and legitimate sensor.
- *Computational Integrity* is achieved through the binding property of the commitment scheme. This creates an unbreakable cryptographic link from the original private inputs

(value and salt) to the final signed commit, ensuring that the computation has not been tampered with.

Furthermore, the combination of provenance and computational integrity provides authenticity (Goal 5). The binding property ensures the data is unaltered, while the signature verification confirms its origin, thereby authenticating the entire data pipeline from sensor to auditor.

Figure B.1 provides a general overview of the use- and misuse cases.

Many of the additional steps do not necessarily require an auditor to detect an attack. Assuming all companies in the supply chain are equally trustworthy, one company claiming to detect fraud by another is not sufficient in some cases. Still, Goal 6 is fulfilled, as the role of the auditor becomes more important, but to assessable degree.

AS	Tampered Artifact	Auditor's Mitigation
AS1	All inputs	A. checks signature in Sensor Key Registry.
AS2	Message, Salt, Commit	A. verifies signature /commit in the journal.
AS3	PCF value of a Proof	A. checks program / Proof Program Registry.
AS4	Proof	A. compares inner proof data with outer journal.
AS5	Commit	A. checks uniqueness via salt and public ledger.

Table 8.1: Summary of the Attack Scenarios, Tampered Artefacts and Mitigation Strategies

Chapter 9

Conclusion and Envisioned Future

9.1 Conclusion

Using *RISC Zero* and Camuda, we designed and implemented a scalable system for carbon footprinting and confidential verification in transport chains with our cloud-native prototype. The architectural considerations of this system were implemented taking into account the context-specific trust and security requirements. In addition, a strong focus was placed on confidentiality (Goal 4), which is essential for collaborative data processing in this context. With the introduction of real-world components, such as a sensor data service, real process data from BPM systems, the latest data exchange standards (iLEAP) and carbon footprint calculation based on GHG, we simulated a realistic scenario.

TRACE was examined in a detailed benchmarking study with regard to performance. For this purpose, four different proof strategies were compared in the experimental setup. All cases were measured and compared in terms of RISC-V cycles, runtime and proof size. Data from 20 entities were used for the measurement.

Our prototype implementation and its evaluation confirm the feasibility of the described system design with strong requirements from the transportation domain. Furthermore, we were able to show in benchmarks that our optimisation approach with lazy signatures means a significant performance improvement, which in turn supports the scalability of carbon footprinting systems using ZKPs. The results further point the way for carbon footprinting in terms of potential cost optimisation through the proposed approaches to automation, which are opposed to manual steps. Here we see clear advantages, also in terms of the auditor’s level, compared to existing industry solutions, such as Siemens SiGreen [14].

9.2 Limitations

In our efforts to build a practical prototype for carbon footprinting, we can confirm feasibility and at the same time, solve one of the most significant limitations of ZKP use by optimising the performance of the proving process. Nevertheless, there are further limitations that must be considered in our approach.

The following limitations relate to the system properties of trust and security. We describe these in ascending order of relevance. Firstly, there is a lack of completeness of the TSP’s contractors’ proofs. It is ensured that the proofs received are processed, but in the implemen-

tation described, there is no way to verify that the TSP has not omitted any contractors. This has serious consequences, as it could allow the PCF to be manipulated by concealing the existence of contractors (and their emissions). Another weakness is that our system does not show a setup in which actual certificates are issued to sensor devices of the contractors by a third party. Although it is assumed that the sensor devices are checked in advance by an auditor, in a real scenario, only a deployed and unchangeable certificate would be considered a trust anchor. Another limitation must be accepted due to our performance optimization. As we have seen, we gain a high level of performance by outsourcing signature verification to the next verifier, but this is associated with less trust. In this use case, this can only be remedied by an additional verification step by the auditor. Another reduction in transparency and security is the lack of real-time evaluation of the proof programs used by the contractors. Although these are stored in the proof program registry, they must be checked manually by the auditor. This would not happen in real time, so a proof program used to create an PCF could be used continuously until the program audit took place. However, incorrect values would already be in circulation, and tracing and revising them would be labour-intensive, even if the swindler could be identified and held accountable.

Although the authenticity of the underlying sensor data for the PCF calculation has been rigorously addressed, the TOC and HOC data exert an equal influence on the final result. Following current industry practice, these figures are often established once and then applied over a set period, creating a significant potential for inaccuracy compared to the dynamic primary data. Therefore, a key limitation of the current prototype is that authenticity verification does not yet extend to these crucial data types.

9.2.1 Threats to Validity Discussion

Our results originate from the described experimental setup (chapter 8). In the following, we will therefore critically examine them to assess their significance. For the assessment, we refer to [4] (Experiments). Aware of the security focus of this validity assessment method, we list internal and external threats to validity.

In the following we discuss threats regarding conclusion, internal, construct and external validity:

Category	Description
Statistical Assumptions	No statistical assumptions regarding sample size are necessary for the simple Pearson correlation coefficient.
Statistical Validity	The data set comprised $n = 20$ elements (TOC and HOC tuples). This sample size is sufficient to reveal strong correlations and trends.
Lack of Expert Evaluation	The system was not independently verified by a life cycle expert for GLEC implementation nor audited by a security expert.
Reliability of Treatment Impl.	All experimental scenarios were conducted equally. The performance-optimised lazy signature verification and identical data sets were consistently used.
Lack of Data Pre-processing	Since the data was synthetically generated, no preprocessing was required for cleaning or handling missing values.

Table 9.1: Threats to construct validity.

Threat	Description
Deficiency of Treatment Setup	All experiments were performed on the same computer to ensure CPU- and memory-related comparability. An additional illustrative experiment was run on a separate, non-randomised data set ($n = 9$).
Ignoring Relevant Factors	It was not investigated whether a computationally weak participant could impair the efficiency of the Composed Proof.
Treatment Design	The experiment focused only on the verification process as the most computationally intense service. A complete investigation would require testing all services or the entire system.

Table 9.2: Threats to internal validity.

Threat	Description
Appropriateness of Data	The data, although derived from sensor data and processed in the Business Process Engine, remains artificial (e.g., max. four TOC/HOC values per element). Random number generation may not reflect real-world ratios of TOC/HOC data.
Experimenter Bias	The experiment was conducted under defined technical conditions. Although subjectivity cannot be fully excluded, the risk of systematic bias is considered low.

Table 9.3: Threats to construct validity.

Threat	Description
Representation of the Setting	A realistic cloud-native environment (Camunda Business Process System) was used. However, a real-world system would involve more participants and scaling. The use of a low-performance computer adds realism.
Context of the Study	The study was conducted in Germany (EU), where environmental issues are emphasized in logistics. The researchers had no direct industry experience; requirements were derived from literature and may not fully reflect actual industry needs.

Table 9.4: Threats to external validity.

9.3 Future Work

With our solution, we have implemented an efficient and verifiable approach (Goal 2) to product carbon accounting based on the iLEAP data standards with nested proofs for the transport sector. Our solution has focused on optimizing the performance of zero-knowledge proofs, as we considered the high computational effort required to generate zero-knowledge proofs to be one of the greatest limitations to practical implementation. In addition, we have defined further essential requirements for our carbon accounting system *TRACE* in chapter 3, which would further improve the performance-optimized version in terms of practicality. In the following, we describe four approaches that should be part of future work in order to enhance the credibility, automatability and customization of verifiable carbon statements.

9.3.1 Proof Program Verification Service

An essential next step in the development of our trace system is the integration of a "Validate Proof Program Service" that enables automated verification of the calculation logic of transmitted proofs. This service is designed to ensure that the calculation of a proof actually meets the requirements for transparency and standard compliance by GHG and PACT specifications.

The core idea is that this service will be operated on the infrastructure of an independent auditor. The input for this is the proof program of the upstream TSP stored in the registry and the associated proof itself. Validation takes place in three consecutive, automated steps:

- Verification of the conformity of the proof program about the GHG/PACT regulations
- Generation of an image ID in a controlled, *RISC Zero* execution environment based on the verified logic.
- Comparison of the image ID with the ID referenced in the proof to ensure authenticity and integrity.

This further development would enable every downstream TSP to check the formal and content validity of received proofs before the actual verification. In the event of a negative validation, further processing could be automatically prevented. This would not only significantly increase

the reliability and trustworthiness of the system but also effectively enforce compliance with global sustainability standards.

9.3.2 Traceability

As we have already stated in our limitations, one of the main weaknesses in terms of tamper resistance is the lack of assurance of completeness. In a chain of supply chain partners, there is no proof that a nested proof contains all the evidence from the actual upstream supply chain. A far-reaching counterfeiting scenario would therefore be to omit a significant issuer when forwarding the nested proof and to pass the proof on to the next party. The final recipient of the proof would thus believe that they were receiving a correct emission value, but this would be lower because it would not reflect the completeness of the supply chains (activities). No problem would arise during verification, as all proofs concealed in the nested proof would have been created truthfully. What is missing, therefore, is automated verifiability of the completeness of the emission assessment, from the origin of the goods to the final destination.

To counter this issue, we suggest adding *traceability* to our approach. Repeatedly, the conflict of transparency and confidentiality appears. On the one hand, the receiver of an PCF proof should be able to verify that indeed the calculated PCF value represents the complete chain of suppliers. On the other hand, the list of suppliers involved in the end-to-end transport service is strictly confidential for business reasons. We agree with the prevailing literature, which suggests the use of a blockchain to solve the problem of traceability. However, we recommend an additional mechanism that controls write permissions on the blockchain without revealing the identity of supply chain actors. Put simply, we assume that the end customer (final destination) knows two things: the origin of their order (company, warehouse) and the destination (place of delivery). Supply chain actors, on the other hand, have authorized knowledge of their upstream and downstream supply chain partners for their delivery order. This is mandatorily documented on the economic side, e.g. contracts document the bilateral relationship between sender and recipient for each section of the supply chain. We use this as a trust anchor. Beyond that, the supply chain actors do not know the pre- or post-transport service of the delivery.

When a customer orders a product from a manufacturer, the manufacturer defines the product's origin in a smart contract using a signed identifier, such as a registered and signed number. The manufacturer then assigns permission to interact with the smart contract to a known transport company, which is responsible for picking up the package. More specifically, the manufacturer grants access to the entity that possesses the hashed contract ID. This hashed ID is exclusively available to the first supply company, which holds the official documents required to collect the goods from the manufacturer. After calculating the carbon emissions using a standard Zero Knowledge Proof approach, the first service provider submits both the result and the corresponding proof image ID to the smart contract. This information is recorded as the first entry in the delivery log. As the goods are handed over to the next provider, the current transport service creates documentation confirming the transfer. In doing so, it also sets access permission to the hash of the specific contract ID that exists between the two service providers. This mechanism enables the receiving party to add its own supply chain footprint and proof image ID to the smart contract. The second service provider repeats this same process, and so on, until the product reaches the end customer. Through the smart contract, the end customer can verify whether the received proof of correctness is linked through an unbroken chain of

predecessors. Ultimately, if the origin of this recursive validation chain can be traced back to the claimed manufacturer, then the proof is considered complete and trustworthy. The use of contract ID-based commitments adds an additional layer of confidentiality (Goal 4) while still allowing for public audit in cases where an external investigation is required.

9.3.3 Selective Disclosure

Our approach highlights the way to confidential, automated verification chains in PCF reporting. Nevertheless, the considerations of Shams et al. point out the potential need for sequential disclosure of specific attributes contained in the PCF [14]. They make the argument that in industry-specific situations, it might be advantageous to have the opportunity to pass only attributes.

9.3.4 Plausibility Checks

A key factor in our motivation was the complexity of the supply chain, which makes it necessary to use the verification mechanism described above to ensure integrity in reporting. However, this complexity and the process-oriented approach makes it difficult to classify the results obtained. In terms of design, this is not actually necessary, as the chained verification ensures the accuracy of the result. However, complete fault tolerance is unlikely given the large number of players and the complexity of the global supply chain. Therefore, the *TRACE* system must be expanded to include plausibility checks. There could be various approaches to this. For example:

1. Scenarios for a transport service could be defined and parameters for worst and best case conditions could be changed
2. It would also be conceivable to use statistical data, where available, to calculate the probabilities of whether deviations from PCFs correspond to the natural distribution or deviate significantly.
3. 1. and 2. could, of course, also be used in combination with ML to learn and detect conspicuous patterns.

In any case, the applied expertise of Life Cycle Assessment (LCA) experts must be incorporated into the development of a plausibility check that expands the system so that effective and realistic plausibility tests can be created.

9.3.5 Verification of Emission Operation Categories

An essential next step toward a fully trustworthy system is the verification of the operation categories (TOC, HOC) used in PCF calculations. This could be achieved in several ways. When these values are derived from exemplary process executions, a straightforward approach is to generate a ZKP for the calculation itself, mirroring the method demonstrated for the PCF. This proof would then become an additional artefact to validate during the main verification phase. Alternatively, for values determined from historical data, verification would require analysis and certification by an auditor or validation against the aforementioned plausibility checks. In either case, establishing a verifiable ground-truth for all operational data is a fundamental requirement.

Bibliography

- [1] Ian Alexander. “Misuse Cases: Use Cases with Hostile Intent”. In: *Software, IEEE* 20 (Feb. 2003), pp. 58–66. DOI: 10.1109/MS.2003.1159030.
- [2] Kirsten Biemann et al. “Treibhausgasemissionen im Transportsektor Leitfaden zur ISO14083”. In: (Mar. 1, 2024), p. 68. ISSN: 2363-832XEps. URL: https://www.umweltbundesamt.de/sites/default/files/medien/479/publikationen/treibhausgasemissionen_im_transportsektor.pdf.
- [3] Smart Freight Centre. *Framework for Logistics Emissions Accounting and Reporting*. Feb. 1, 2025. 172 pp. ISBN: 978-90-833629-0-8. URL: https://smart-freight-centre-media.s3.amazonaws.com/documents/GLEC_FRAMEWORK_v3.1_March_2025_1-3.pdf (visited on 08/06/2025).
- [4] Daniela S Cruzes and Lotfi ben Othmane. “Threats to validity in empirical software security research”. In: *Empirical research for software security*. CRC Press, 2017, pp. 275–300.
- [5] Uriel Feige, Amos Fiat, and Adi Shamir. “Zero-knowledge proofs of identity”. In: *Cryptography 1* (June 1988), pp. 77–94. DOI: 10.1007/BF02351717.
- [6] Till Gramberg, Thomas Bauernhansl, and Andreas Eggert. “Disruptive Factors in Product Portfolio Management: An Exploratory Study in B2B Manufacturing for Sustainable Transition”. In: *Sustainability* 16.11 (Jan. 2024). Number: 11 Publisher: Multidisciplinary Digital Publishing Institute, p. 4402. ISSN: 2071-1050. DOI: 10.3390/su16114402. URL: <https://www.mdpi.com/2071-1050/16/11/4402> (visited on 12/09/2024).
- [7] Jonathan Heiss et al. “Advancing Blockchain-based Federated Learning through Verifiable Off-chain Computations”. In: *2022 IEEE International Conference on Blockchain (Blockchain)* (2022), pp. 194–201. DOI: 10.1109/Blockchain55522.2022.00034.
- [8] Jonathan Heiss et al. “Verifiable Carbon Accounting in Supply Chains”. In: *IEEE Transactions on Services Computing* 17.4 (2024), pp. 1861–1874. DOI: 10.1109/TSC.2023.3332831.
- [9] Florian A. Jaeger et al. “Challenges and requirements of exchanging Product Carbon Footprint information in the supply chain”. In: *E3S Web of Conferences* 349 (2022). Publisher: EDP Sciences, p. 07005. ISSN: 2267-1242. DOI: 10.1051/e3sconf/202234907005. URL: https://www.e3s-conferences.org/articles/e3sconf/abs/2022/16/e3sconf_1cm2022_07005/e3sconf_1cm2022_07005.html (visited on 10/18/2024).
- [10] Gaurav Jha, Shatrughan Soren, and Kapil Deo Mehta. “Carbon footprint assessment with LCA methodology”. In: *LCA based carbon footprint assessment*. Springer, 2021, pp. 1–34.

-
- [11] Walter Klöpffer. “Life cycle assessment”. In: *Environmental Science and Pollution Research* 4.4 (Dec. 1, 1997), pp. 223–228. ISSN: 1614-7499. DOI: 10.1007/BF02986351. URL: <https://doi.org/10.1007/BF02986351> (visited on 08/04/2025).
- [12] Nuri C Onat et al. “Driving Sustainable Business Practices With Carbon Accounting and Reporting: A Holistic Framework and Empirical Analysis”. In: *Corporate Social Responsibility and Environmental Management* 32.2 (2025), pp. 2795–2814.
- [13] Imad Raad et al. “Zero-Knowledge Proof”. In: *Trends in Data Protection and Encryption Technologies* 17.4 (2024), pp. 25–30. DOI: 10.1007/978-3-031-33386-6_6. URL: https://link.springer.com/chapter/10.1007/978-3-031-33386-6_6.
- [14] Saad Bin Shams et al. “Trustworthy Supply Chain Exchange for Product Carbon Footprint”. In: *2023 IEEE International Conference on Artificial Intelligence, Blockchain, and Internet of Things (AIBThings)*. Sept. 2023, pp. 1–6. DOI: 10.1109/AIBThings58340.2023.10292452. URL: <https://ieeexplore.ieee.org/document/10292452/> (visited on 06/20/2025).
- [15] Siemens. *Digital Ecosystems: A Data-Driven Approach To Decarbonizing Supply Chains*. June 15, 2022. URL: <https://hbr.org/sponsored/2022/06/digital-ecosystems-a-data-driven-approach-to-decarbonizing-supply-chains> (visited on 08/04/2025).
- [16] Ron Steinfeld. “Post-Quantum Zero-Knowledge Proofs and Applications”. In: *ASIA CCS 23: ACM ASIA Conference on Computer and Communications Security* (July 2023), pp. 1–1. DOI: 10.1145/3591866.3593075. URL: <https://dl.acm.org/doi/10.1145/3591866.3593075>.
- [17] Janwillem Swalens et al. “zkStream: a Framework for Trustworthy Stream Processing”. In: *Proceedings of the 25th International Middleware Conference*. Middleware ’24. Hong Kong, Hong Kong: Association for Computing Machinery, 2024, pp. 252–265. ISBN: 9798400706233. DOI: 10.1145/3652892.3700763. URL: <https://doi.org/10.1145/3652892.3700763>.
- [18] Mathias Weske. *Business Process Management: Concepts, Languages, Architectures*. en. Berlin, Heidelberg: Springer Berlin Heidelberg, 2024. ISBN: 978-3-662-69518-0. DOI: 10.1007/978-3-662-69518-0. URL: <https://link.springer.com/10.1007/978-3-662-69518-0> (visited on 07/31/2025).
- [19] World Bank. *Digital Monitoring, Reporting, and Verification Systems and Their Application in Future Carbon Markets*. Washington, DC: World Bank, May 31, 2022. DOI: 10.1596/37622. URL: <https://hdl.handle.net/10986/37622> (visited on 07/29/2025).

Appendix A

Abbreviations

BPM Business Process Management

BPMN Business Process Model and Notation

DSLs Domain-Specific Languages

GHG Greenhouse Gas

GLEC Global Logistics Emissions Council

HOC Hub Operation Category

iLEAP Integrating Logistics Emissions and PCFs

LCA Life Cycle Assessment

MRV Monitoring-Reporting-Verification

PACT Partnership for Carbon Transparency

PCF Product Carbon Footprint

PFT Product Footprint Template

TCE Transport Chain Element

TOC Transport Operation Category

TSP Transport Service Provider

VCA Verifiable Carbon Accounting

VCs Verifiable Credentials

ZKP Zero-Knowledge Proof

zk-SNARK Succinct Non-Interactive Arguments of Knowledge

zk-STARK Scalable Transparent Arguments of Knowledge

zkVM Zero-Knowledge Virtual Machine

Appendix B

Documents, Data Examples and AI Generated Content

B.1 Proving Document

Proving Document

```
1 {
2   "productFootprint": {
3     "id": "e4ad7f00-875c-4f20-bbee-d586c3ce62c4",
4     "specVersion": "2.0.0",
5     "version": 0,
6     "created": "2025-07-26T03:40:10.332682",
7     "status": "Active",
8     "companyName": "Amazing Company 2",
9     "companyIds": [
10      "urn:epcids:1fe5f348-9ef2-461a-877b-2c062501edd7"
11    ],
12    "productDescription": "Logistics emissions related to shipment with ID SHIP_11777410-3f49-441f-af0a-1c7695a2a029",
13    "productIds": [
14      "urn:pathfinder:product:customcode:vendor-assigned:457a5362-8cc0-4c19-9a19-cdd65006c834"
15    ],
16    "productCategoryCpc": 7290,
17    "productNameCompany": "Shipment with ID SHIP_11777410-3f49-441f-af0a-1c7695a2a029",
18    "pcf": null,
19    "comment": "",
20    "extensions": [
21      {
22        "specVersion": "2.0.0",
23        "dataSchema": "https://api.ileap.sine.dev/shipment-footprint.json",
24        "data": {
25          "mass": 14164.714410018489,
26          "shipmentId": "SHIP_11777410-3f49-441f-af0a-1c7695a2a029",
27          "tces": [
28            {
29              "tceId": "4ca462e5-ba9d-4d30-bf07-5a7be313eadc",
30              "prevTceIds": [],
31              "hocId": null,
32              "tocId": "201",
33              "shipmentId": "SHIP_11777410-3f49-441f-af0a-1c7695a2a029",
34              "mass": 14164.714410018489,
35              "co2eWTW": null,
36              "co2eTTW": null,
37              "transportActivity": null,
38              "distance": {
39                "actual": 11.94,
40                "gcd": null,
41                "sfd": null
42              }
43            },
44            {
45              "tceId": "ea79c04c-244d-43b1-893d-60433f0a0ed9",
46              "prevTceIds": [
47                "4ca462e5-ba9d-4d30-bf07-5a7be313eadc"
```

```

48         ],
49         "hocId": "102",
50         "tocId": null,
51         "shipmentId": "SHIP_11777410-3f49-441f-af0a-1c7695a2a029",
52         "mass": 14164.714410018489,
53         "co2eWTW": null,
54         "co2eTTW": null,
55         "transportActivity": null,
56         "distance": null
57     },
58     {
59         "tceId": "644fd33f-1db9-40b1-95e4-3e461f6abb58",
60         "prevTceIds": [
61             "4ca462e5-ba9d-4d30-bf07-5a7be313eadc",
62             "ea79c04c-244d-43b1-893d-60433f0a0ed9"
63         ],
64         "hocId": null,
65         "tocId": "202",
66         "shipmentId": "SHIP_11777410-3f49-441f-af0a-1c7695a2a029",
67         "mass": 14164.714410018489,
68         "co2eWTW": null,
69         "co2eTTW": null,
70         "transportActivity": null,
71         "distance": {
72             "actual": 132.88,
73             "gcd": null,
74             "sfd": null
75         }
76     },
77     {
78         "tceId": "4f42314b-53d3-47ba-8ce4-ca4e0f7503cb",
79         "prevTceIds": [
80             "4ca462e5-ba9d-4d30-bf07-5a7be313eadc",
81             "ea79c04c-244d-43b1-893d-60433f0a0ed9",
82             "644fd33f-1db9-40b1-95e4-3e461f6abb58"
83         ],
84         "hocId": "103",
85         "tocId": null,
86         "shipmentId": "SHIP_11777410-3f49-441f-af0a-1c7695a2a029",
87         "mass": 14164.714410018489,
88         "co2eWTW": null,
89         "co2eTTW": null,
90         "transportActivity": null,
91         "distance": null
92     }
93 ]
94 }
95 }
96 ]
97 },
98 "tocData": [
99     {
100         "tocId": "201",
101         "certifications": [
102             "GLECv3.1"
103         ],
104         "description": "Electric Van - Urban Delivery",
105         "mode": "road",
106         "loadFactor": "0.65",
107         "emptyDistanceFactor": "0.05",
108         "temperatureControl": "None",
109         "truckLoadingSequence": "Optimized Route",
110         "airShippingOption": null,
111         "flightLength": null,
112         "energyCarriers": [
113             {
114                 "energyCarrier": "Electricity",
115                 "relativeShare": "1.0",
116                 "emissionFactorWTW": "30 gCO2e/tkm",
117                 "emissionFactorTTW": "0 gCO2e/tkm"
118             }
119         ],
120         "co2eIntensityWTW": "30 gCO2e/tkm",
121         "co2eIntensityTTW": "0 gCO2e/tkm",
122         "transportActivityUnit": "vkm"
123     },
124     {
125         "tocId": "202",
126         "certifications": [
127             "ISO14083:2023",
128             "GLECv2"
129         ],
130         "description": "Air Freight - International Cargo",

```

```

131     "mode": "air",
132     "loadFactor": "0.70",
133     "emptyDistanceFactor": "0.02",
134     "temperatureControl": "Refrigerated +2C to +8C",
135     "truckLoadingSequence": "None",
136     "airShippingOption": "Dedicated Cargo Aircraft",
137     "flightLength": "Long Haul (>4000km)",
138     "energyCarriers": [
139       {
140         "energyCarrier": "Jet Fuel (Kerosene)",
141         "relativeShare": "1.0",
142         "emissionFactorWTW": "700 gCO2e/tkm",
143         "emissionFactorTTW": "650 gCO2e/tkm"
144       }
145     ],
146     "co2eIntensityWTW": "700 gCO2e/tkm",
147     "co2eIntensityTTW": "650 gCO2e/tkm",
148     "transportActivityUnit": "tkm"
149   }
150 ],
151 "hocData": [
152   {
153     "hocId": "102",
154     "passhubType": "Logistics Hub",
155     "energyCarriers": [
156       {
157         "energyCarrier": "Electricity",
158         "relativeShare": "0.3",
159         "emissionFactorWTW": "25 gCO2e/MJ",
160         "emissionFactorTTW": "0 gCO2e/MJ"
161       },
162       {
163         "energyCarrier": "Diesel",
164         "relativeShare": "0.7",
165         "emissionFactorWTW": "95 gCO2e/MJ",
166         "emissionFactorTTW": "73 gCO2e/MJ"
167       }
168     ],
169     "co2eIntensityWTW": "95 gCO2e/MJ",
170     "co2eIntensityTTW": "73 gCO2e/MJ",
171     "hubActivityUnit": "number of vehicles serviced"
172   },
173   {
174     "hocId": "103",
175     "passhubType": "Multi-modal Energy Hub",
176     "energyCarriers": [
177       {
178         "energyCarrier": "Electricity",
179         "relativeShare": "0.4",
180         "emissionFactorWTW": "25 gCO2e/MJ",
181         "emissionFactorTTW": "0 gCO2e/MJ"
182       },
183       {
184         "energyCarrier": "HVO100",
185         "relativeShare": "0.35",
186         "emissionFactorWTW": "20 gCO2e/MJ",
187         "emissionFactorTTW": "15 gCO2e/MJ"
188       },
189       {
190         "energyCarrier": "CNG",
191         "relativeShare": "0.25",
192         "emissionFactorWTW": "55 gCO2e/MJ",
193         "emissionFactorTTW": "50 gCO2e/MJ"
194       }
195     ],
196     "co2eIntensityWTW": "30 gCO2e/MJ",
197     "co2eIntensityTTW": "20 gCO2e/MJ",
198     "hubActivityUnit": "energy delivered (MJ)"
199   }
200 ],
201 "signedSensorData": [
202   {
203     "tceId": "4ca462e5-ba9d-4d30-bf07-5a7be313eadc",
204     "camundaProcessInstanceKey": "2251799813685541",
205     "camundaActivityId": "Activity_10zebyt",
206     "sensorkey": "-----BEGIN PUBLIC KEY-----\n
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAs2iVoqS8SGGFh8oqhDjG\n
nFeo+oWiO0/E++kFhuC01qzOwL2xXTMT6JF1Y6LY6xHGoxSF8Xgvu0Ycn+FL8UZA\n
neH5G2kpvilMfxY7B5Ly3rqs6E4amuCHZzrrdwjWlaLbnJzY58uJlDSMbHqtiD3/\n
nv8CG1X9gMCYyV48IYV5fq2P65WslXZoQLk7VeqPhPCKU060c47WAEZ3gqYnuLPd\n
nAmUMV8EVAh6OXFMldflkoz1Svy6m2ReEp2FiTiBvt3adqQOX8HXZVnPWwJUilC\n
nFmCNgKAG5fxffLpACeRwJS8Branka01IwiQe1BUkxsEO81J4oTH9mlbAi6F0GKG\
ntwIDAQAB\n-----END PUBLIC KEY-----\n",
207     "signedSensorData": "0gnxUwMzbAb25/Z8ioiFZIZQZPA3h7QtrMMTyxEx2JHUSV1txaFULVs66XQy/+TsgJa40xFkp0/

```

```

q6ixX48YMEUKW6XdaFZs/UI4K0xzkjiGvwW6V8yTQz5j/MDaB1EmKcc3M/tW4dLoZy4VJqFmKu/GQj4jxWTRye1+rFuAKf3/JgUbg/31
PLm92UWLKz2n0+czPt8nVS0F0HF6dF0Z/GJd8aWy+t5aiFjgelwiHr05E9UYr/jHM60PUPgv/vmLdwLAZcjinM/yiHmh/29
cnwk3oYJrP407YwSeTqb3Dv2Xp/iZ1Ha3KQZVEytFCF550+vVvAsMoWIVv2Qq+aHbUA==" ,
208 "sensorData": {
209   "distance": {
210     "actual": 11.94,
211     "gcd": null,
212     "sfd": null
213   }
214 },
215 "salt": "Geosys2W8Pa9mR0nmMvhirtrKpthysAD0",
216 "commitment": "W4WYz0sFCDRj0AY0ZhhxTaxCumFcXr6G/D8lr0JVWHY="
217 },
218 {
219   "tceId": "644fd33f-1db9-40b1-95e4-3e461f6abb58",
220   "camundaProcessInstanceKey": "2251799813685541",
221   "camundaActivityId": "Activity_0t601wy",
222   "sensorkey": "-----BEGIN PUBLIC KEY-----\n
nMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAvZaIMZfomUgiAr80PfGL\nnpXsA3usv0yn/QiHiSTXiAYMBsJDi8S8QY+
EeoY4eW2w9A5tvE6kTHhgdR9FdEDD8\nnw4mX+WH3ejsVfGUjU1M0rT9/I883gj0XF39K0heqKwBxo1btyMYInSnaI4EUN1MU\
nSVoJcwqpMymI+0LSsCUYcmHL5mwwKQ7Ixe10hjsZGdan4k88A2dxP2AhMUx0Aaaf\
nJ51bSK86sfGy5Uqc8qwmjmeFuE0hBPFU3bMvm1tRKsJEdCPXk/kbz4Sq68UCRnNz\
nOR6SsMQBcyjhSJ1DMiV8VCuekFx4vDcKrfEiM6RYo0e7jRlWzR9qlF1jmUMXSSVh\nnewIDAQAB\n-----END PUBLIC KEY-----\n",
223   "signedSensorData": "rDb0512xvZXqMEqPTNFzMXw9StG9U2w0ThB09Hkp0N4pnWAXZTYxkNMOB11+yQ7SesNFAPt7UWXMeIrCXk/
il66W7yF5KU8/vudioCqG9iKZ40Q1w1VpU3JtXa1w1Et18d0ateNza+g8XWRfiIgFlu5jyrCKQocynpNDYN+
H489CtoD9R1qSDikYT5Uf6iNytJ21Afz7LAZ1/ULhQXvrHia7tyN3YketJuaR7HrR0XHy/
g2gaBrmsVZJ10NFf0ecYUVZ8KtkhUULS10LVJtu5Vyt19YA5X9MhYgPlkF/QkWBmehZDVtfFkC1zb7U45F7beuT/tmhxUeGZ9YRBvpA
==",
224   "sensorData": {
225     "distance": {
226       "actual": 132.88,
227       "gcd": null,
228       "sfd": null
229     }
230   },
231   "salt": "kzAP3ifyy4JpRcJrWB6bpIAjit9kcafh",
232   "commitment": "5UZD70+vROVPvSTCup0XuZiBz0FXIURZqQQgQlk0aTo="
233 }
234 ]
235 }

```

B.2 ProofResponse

Example: Proof Response from Proving Service

```

1 {
2   "productFootprintId": "45d17bd3-2506-433f-864b-0f139492734a",
3   "proofReceipt": "eJyNVE1v2zAM/SsGz7Y127Jk3xq0QYs0GLBhww5FgcCQbTIRYMu0JaVpivz3yU...",
4   "pcf": 2847.3421,
5   "imageId": "4a5284543936e6d1f6e1e672ab583706b46d9be7a752f2acd3f41e91704d7d7e",
6   "proofReference": ""
7 }

```

B.3 Proving Service Pseudo Code

```

1 FUNCTION MAIN_PROVING_LOGIC(proving_document):
2   // Extract previous proofs for chaining
3   previous_proofs = proving_document.proof
4   proving_document.proof = EMPTY_LIST
5
6   // Build execution environment
7   env_builder = CREATE_EXECUTOR_ENV_BUILDER()
8   WRITE_PROVING_DOCUMENT(env_builder, proving_document)
9
10  // Process previous proofs with cryptographic verification
11  FOR EACH proof IN previous_proofs DO
12    VERIFY_PROOF_RECEIPT(proof.receipt, proof.imageId)
13    VERIFY_CRYPTOGRAPHIC_SIGNATURES(proof.signatureContainers)
14    ADD_PROOF_ASSUMPTION(env_builder, proof.receipt)
15    ADD_VERIFICATION_DATA(env_builder, proof.imageId, proof.journal)

```

```

16  END FOR
17
18  executor_env = BUILD_ENVIRONMENT(env_builder)
19
20  // Execute zkVM proving with error handling
21  prover = GET_DEFAULT_PROVER()
22  TRY:
23      prove_info = EXECUTE_PROVING(prover, executor_env, GUEST_LOGIC_ELF)
24      receipt = prove_info.receipt
25  CATCH proving_error:
26      RETURN None
27
28  // Verify receipt and extract results
29  TRY:
30      VERIFY_RECEIPT(receipt, GUEST_LOGIC_ID)
31      (pcf_value, signature_containers) = DECODE_JOURNAL(receipt.journal)
32  CATCH verification_error:
33      RETURN None
34
35  // Serialize and encode proof receipt
36  encoded_receipt = ENCODE_PROOF_RECEIPT(receipt)
37
38  // Return structured proof response
39  RETURN ProductProof(
40      productFootprintId: proving_document.productFootprint.id,
41      proofReceipt: encoded_receipt,
42      pcf: pcf_value,
43      imageId: ENCODE_GUEST_ID(GUEST_LOGIC_ID),
44      proofReference: GENERATE_REFERENCE()
45  )
46 END FUNCTION

```

Listing B.1: Main Proving Logic Implementation

```

1  FUNCTION MAIN_GUEST_LOGIC():
2      // Initialize transport PCF and signature containers
3      transport_pcf = 0.0
4      sig_containers = EMPTY_LIST
5
6      // Read proving document from host
7      previous_proof_verification_data = ENV.READ()
8      proving_document = ENV.READ()
9
10     // Process previous proofs if any exist (proof composition)
11     FOR EACH container IN previous_proof_verification_data DO
12         // Verify previous proof within zkVM and extract PCF
13         VERIFY_JOURNAL_AUTHENTICITY(container.imageId, container.journal)
14         pcf_value = DECODE_JOURNAL(container.journal)
15         transport_pcf += pcf_value
16     END FOR
17
18     // Process current company's TCEs (Transport Chain Elements)
19     tces = proving_document.productFootprint.extensions[0].data.tces
20
21     FOR EACH tce IN tces DO
22         // Process Transport Operations (TOCs)
23         IF tce.tocId IS_NOT_NULL AND tce.distance IS_NOT_NULL THEN
24             emission_factor = LOOKUP_TOC_EMISSION_FACTOR(tce.tocId)
25             emissions = tce.mass * emission_factor * tce.distance.actual
26             transport_pcf += emissions
27
28             // Verify sensor data hash commitments if available
29             IF sensor_data_exists_for(tce) THEN
30                 VERIFY_HASH_COMMITMENT(tce.sensorData, tce.salt, tce.commitment)
31                 sig_containers.ADD(CREATE_SIGNATURE_CONTAINER(tce.signedData))
32             END IF
33         END IF
34
35         // Process Handling Operations (HOCs) - no distance factor
36         IF tce.hocId IS_NOT_NULL THEN
37             emission_factor = LOOKUP_HOC_EMISSION_FACTOR(tce.hocId)
38             emissions = tce.mass * emission_factor
39             transport_pcf += emissions
40         END IF
41     END FOR
42
43     // Commit results to zkVM journal for proof generation
44     COMMIT_TO_JOURNAL(transport_pcf, sig_containers)
45
46     RETURN transport_pcf
47 END FUNCTION

```

Listing B.2: Guest Program Logic with Proof Composition and TCE Processing

B.4 Use/Misuse overview

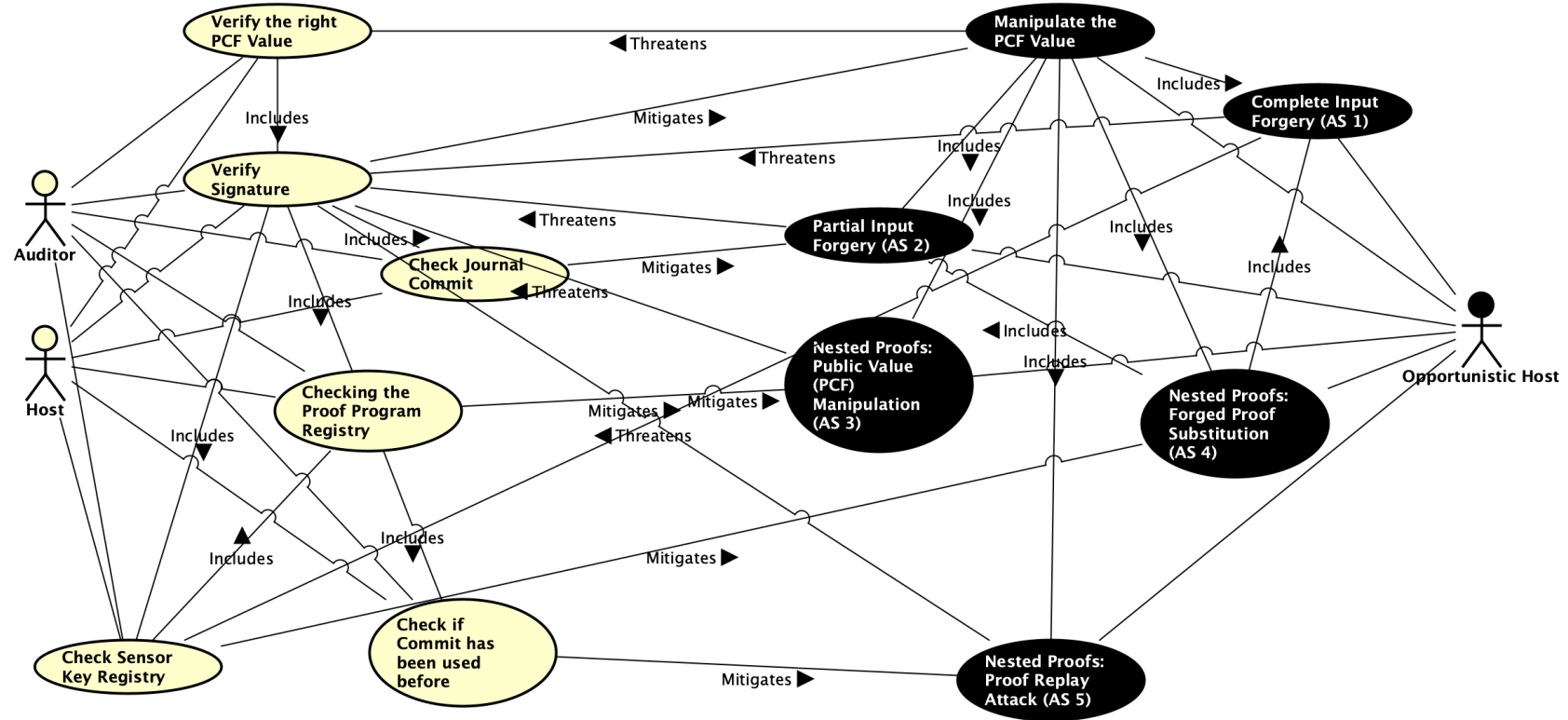


Figure B.1: General overview. Use/misuse-case diagram. Use-case elements appear on the left, the misuse case elements on the right. Leaned on [1]

B.5 AI Generated Prompts

User: Can u write an abstract for my report?

LLM Response:

Looking at your TRACE report, I can write an abstract that captures the key elements of your work. Here's a proposed abstract:

Abstract The increasing regulatory pressure and public demand for transparent carbon accounting in global supply chains presents a fundamental challenge: how to verify Product Carbon Footprint (PCF) calculations while protecting sensitive business information. This report presents TRACE (Trusted Real-Time Assessment of Chained Emissions), a prototypical implementation of a verifiable carbon accounting system for transport chains that resolves this tension through the integration of Zero-Knowledge Proofs (ZKPs) and business process management.

TRACE addresses the limitations of traditional trust-based carbon reporting systems by enabling mathematical verification of PCF calculations without disclosing confidential operational data. The system implements a process-centric approach using BPMN-based workflow orchestration through Camunda, combined with cryptographic verification using RISC Zero's zkVM. By adhering to established industry standards including iLEAP, GLEC, PACT, and ISO 14083, TRACE ensures interoperability while introducing verifiable computation capabilities.

The architecture employs a microservice design with clearly separated private and public domains, enabling companies to maintain control over sensitive data while participating in transparent carbon accounting networks. A key innovation is the implementation of lazy signature verification, which significantly reduces the computational overhead of proof generation—a critical bottleneck in ZKP systems. Performance benchmarking demonstrates that nested proof composition scales linearly with chain depth while maintaining constant proving time, making the approach viable for real-world transport scenarios with multiple participants.

Evaluation results confirm that TRACE successfully fulfills its core objectives: enabling standards-compliant PCF calculations, providing cryptographic verification without compromising business confidentiality, and achieving practical performance for multi-party supply chains. While certain limitations remain, particularly regarding completeness verification and the need for trusted auditors in specific scenarios, TRACE demonstrates the feasibility of transitioning from trust-based to mathematically verifiable carbon accounting systems in the transportation sector.

This abstract follows the standard structure of background/problem, solution, methodology, results, and implications while staying within typical length constraints (around 250-300 words). Would you like me to adjust the length, emphasis, or technical detail level?

B.6 Project Contributions

The following table dovetails who did what in our project. The list of contributors is order from most to least *time spent* and *impact made*. "Rework" means a complete overhaul with drastic changes to content.

Code	Main Participants
Camunda Service	Thang, Frederic, Reine
Proving Service	Reine, Thang, Lucas, Ramon
Verifier Service	Ramon, Thang, Reine
Sensor Data	Greta, Ramon, Thang
Kafka	Thang, Reine, Lucas
PCF Registry	Felix
Proof Registry	Yunusemre
Sensor Key Registry	Thang
Deployment	Greta, Thang
End-to-End-Integration	Thang
GitHub Documentation	Thang
Report	
Motivation	Greta, Lucas
Background	
BPM	Thang, Lucas, Greta
ZKP	Ramon, Lucas, Greta
Risc0 and zkVM	Ramon, Lucas, Greta
iLeap and Data Standards	Frederic, Greta
Related Work	Ramon, Lucas
Goals/Requirements and Assumptions	Frederic, Thang, Greta
System Design	
Scenario	Frederic, Greta
Architecture Overview	Frederic, Greta
Service Components	Thang, Greta
PCF Registry Part	Felix, Thang (Rework)
Component Interaction	Frederic
Integrating and extending existing standards	Frederic
Auditor Role	Frederic
Setup and Configuration	Frederic, Thang
Trust System	Lucas
Implementation	Thang, Frederic (Rework)
Benchmark and Methodology	Reine
Evaluation: Performance	Frederic, Reine
Evaluation: Lazy Signature	Ramon, Frederic (Rework)
Conclusion	Lucas
Future Work and Limitations	Lucas
Review	Frederic, Greta, Lucas, Thang
Abbreviations	Ramon
Bibliography	Ramon
Presentations	
Concept and Planning	Lucas, Frederic
Slides	Lucas, Frederic
Benchmark Plots	Reine
Learning Text and Presenting	Frederic, Lucas
Feedback	Frederic, Lucas, Thang